

=====

PlayList : spring boot full course

channel : india free internship

youtube channel <https://www.youtube.com/@IndiaFreeInternship>

source Code: [spring boot source Code](#)

notes Link : <https://github.com/indiafreeinternship/NOTES/tree/main/spring%20boot%20notes>

live notes and live coding

=====

SESSION 1

1. SPRING FRAMEWORK

2. SPRING BOOT

3. SRPING DATA JPA

4. SPRING WEB MVC

5. SPRING REST

6. SPRING SECURITY

7. MICROSERVICE

8. SPRING BATCH

9. SPRING SCHEDULAR

=====

SESSION 2

=====

Technolgies : webPage(servlets) + Database(JDBC/JPA) + Email (Java Mail) + Message(JMS)

Limitations:

- Start coding from line zero
- Development takes more time
- More chance of error
- Design pattern applied manually

Framework:

- Ready-made structure
- In-built technologies
- Design patterns already implemented

RAD => Rapid Application Development

- Fast coding
- Less Error

Spring Framework:

- used for Web Application or Distributed appliation
- It has own Apis
- It Follow Container System

Spring Container

- Find/Scan Classes[spring bean]
- Create Object
- provide DATA
- Link Object based on Relation
- Finally Destroy

Depedency Injection (DI)

Depedency Injection is a concept/theory which is implemented by spring IoC (Inversion of control) also called as spring container.

Theory	programming
oops	core java
ORM	HIbernate with JPA
DI	Spring Ioc(Inversion of control)

session 3

=====

Application

=====

3 Type of application

- (i) Web Application**
- (ii) Distributed Application**
- (iii) Microservices Application**

(i) Web Application : Collection of web pages run under webserver.

Application :- n services
Project :- n Module

eg : Gmail Application

User(Register and Login) , Inbox, Sent, Drfats, Setting, etc

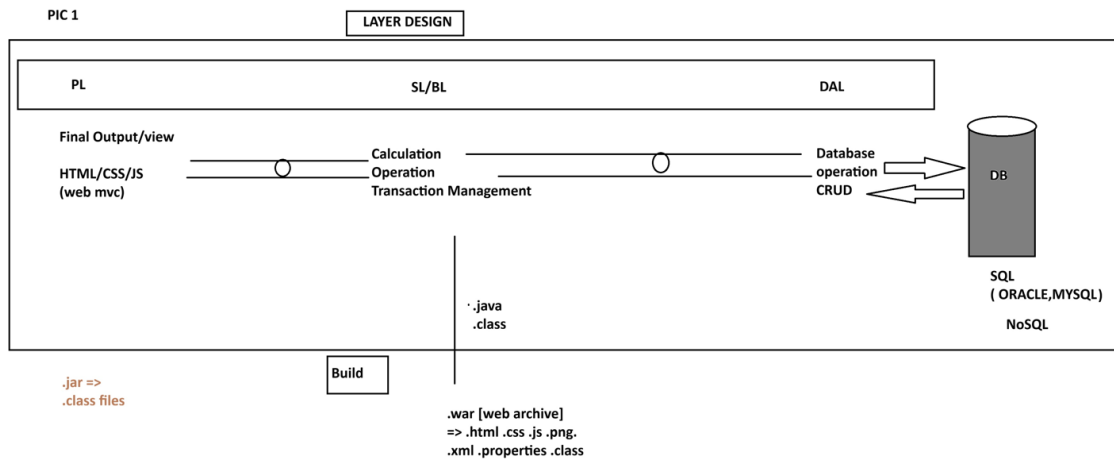
Amazon Application

User(Register and Login) , search, Cart, Payment, ...etc

=> Implementation of service / module: - This is implemented using 3 Layers Design.

- 1. DAL : Data Access Layer (Database Operations)**
- 2. SL : Service Layer (Calculation/Operations/txn mngmt)**
- 3. PL : Presentation layer (view /Dynamic web pages/MVC)**

PIC 1



(ii) Distributed Application : An Application that runs at mutiple devices is called as distributed appliation.
xyz Bank application

This appliation can be accessed in different ways. Example.

eg:

- 1. Manually Process**
- 2. NetBanking (Web App)**
- 3. Mobile Banking**
- 4. IVR**
- 5. ATM / DC**
- 6. 3rd party app (Gpay, PayPhone..etc)**

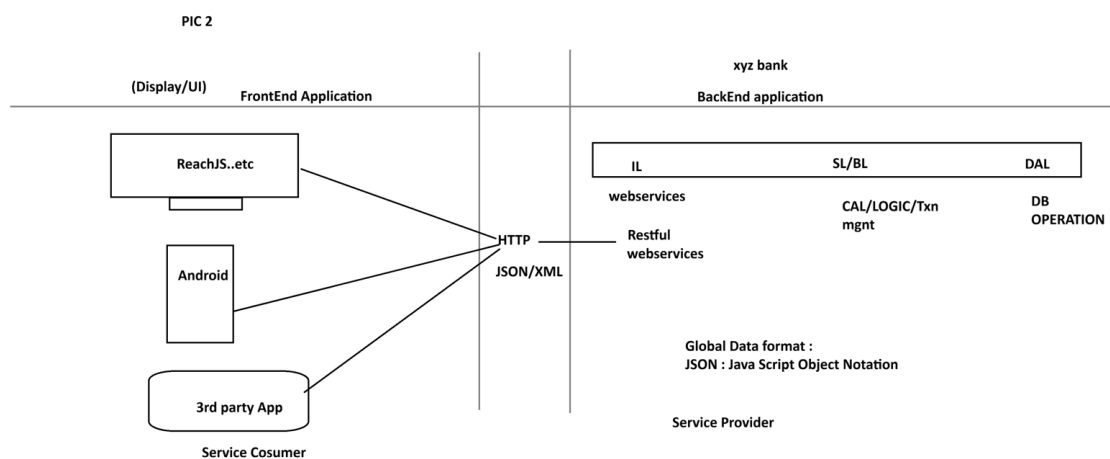
.....

-But here business logics exist at backend application.

- Fontend applications are implemented using Angular, Android...etc

- App connected using HTTP Protocol using Global Data Format (JSON/XML)

PIC 2

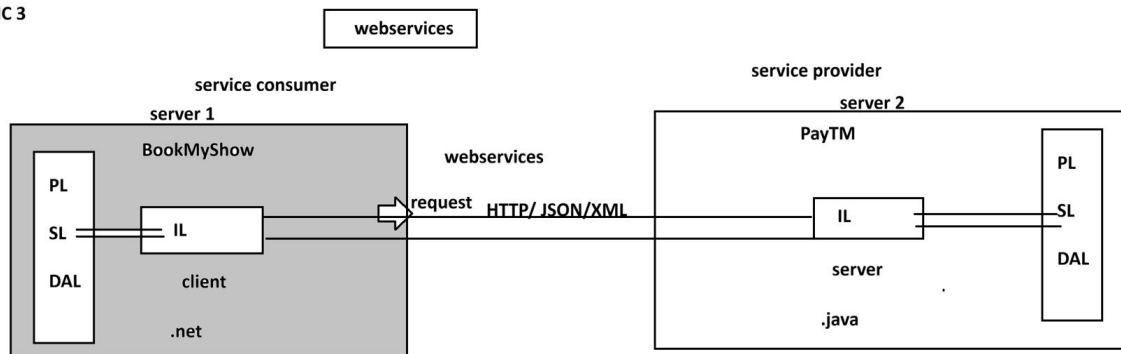


WebServices:

Intergration of multiple (at least two) applications which runs on different servers and implemented using different language (java, .net, php, python....etc)

PIC 3

PIC 3



session 4

=====
=====
SPRING FRAMEWORK
=====

CORE

-
1. DI and IOC
 2. Container Types
 3. Annotation Configuration
 4. @ComponentScan
 5. @Component
 6. @Value
 7. Java Configuration
 8. @Configuration
 9. @Bean
 10. @PropertySource
 11. Environment
 12. Lifecycle methods
 13. Runners
 14. @ConfigurationProperties
 15. Properties file
 16. YAML File
 17. Profiles
 18. Lombok API

=====
spring core chapter 1
=====

Core:

This module/chapter gives all rules and guidelines to work with spring container.

1. DI and IOC

=====

Spring Container :- It takes care of object.

1. Find/Scan our Classes(spring bean)
2. Create Object
3. Provide Data
4. Link Object
5. Destroy the Object

=>For this programmer has to provide two inputs-

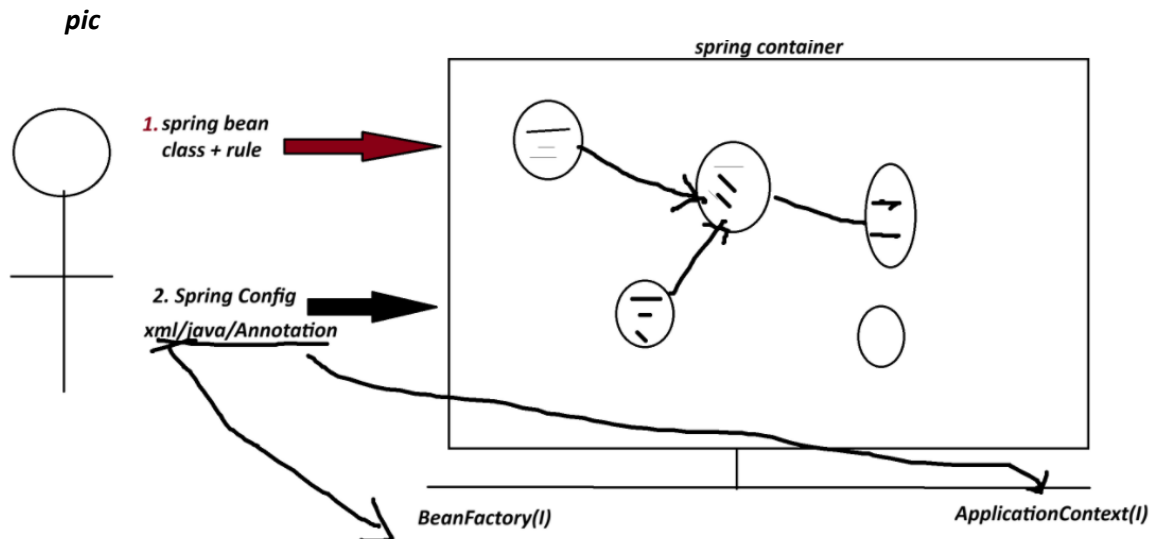
1. Spring Bean : class+rules given by spring container.
2. Spring Configuration : This is main input contains details like object name, relation with objects, data....etc.

xml/java/Annotation

**** Spring Container 2 Types:**

1. BeanFactory (Old container) : support only XML configuration file.

2. **ApplicationContext (New container) :** support xml/java/Annotation configuration.



=====

Dependency Injection (DI)

<i>Theory</i>	<i>programming</i>
<i>OOPs</i>	<i>java</i>
<i>ORM</i>	<i>Hibernate with JPA</i>
<i>DI</i>	<i>Spring IoC(Spring Container) -> Inversion of Control</i>

=> **Depedency** :- An instance variable (Field) exist inside a class (spring bean)

1. **Primitive Type Dependency (PTD)** :- If a variable is created using (8+1) datatype.
(byte, short, int, long, float, double, boolean, char +string)
(Their wrapper classess too)

2. **Collection Type Dependency (CTD)** :- If a variable created using(4) types:
(java.util.package)
(List,Map,Set and Properties)

3. **Reference Type Dependency (RTD)** :- If a variable is creatd using other class/interface

Injection(4)

=====

=> It give/provide/assign existed provide data to Depedency(Inside object)

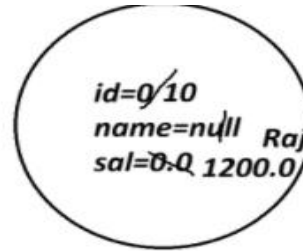
1. **Setter Injection (SI)** :- provide data to variable using its set method(setter).

2. **Constructor Injection(CI)**: provide data creating object using parameterized constructor.

3. **LookUpMethod Injection(LMI)** :- Spring bean Scope
[When parent bean is sigleton and child bean is prototype]

4. **Interface Injection(II)** :- Not Exist in Spring F/W.

```
class Employee{
    int id;
    String name;
    double sal;
}
```



Employee e=new Employee()

```
class Product{
    int id; //PTD
    String name; //PTD
    Boolean exist; //PTD
```

```
class Vendor{}

interface Service{}
```

```
List<String> models; //CTD
Map<String,Integer> data; //CTD
```

```
Vendor vob; //RTD
Service sob; //RTD
```

```
}
```

SESSION 5

=====

1. Spring Bean

2. STS Installation

Spring Bean

=====

-It is class that follow rules given by Spring Container.

Rule

- class must be public Type.
- Recomended to keep in a package (basePackage Rule)

ex :

com.app.service

com.app.controller

- Fields(Variables) are option, if required recomended to keep private.

```
private Integer empId;
```

```
private String empName;
```

- If variable present in class,

=>define default constructor with setter/getter

=>(or/and) Parameterized Constructor.

ex:

```
public class Employee{  
    private Integer id;  
    private String name;  
    public Employee(){  
    }  
    public void setId(Integer id){  
        this.id=id;  
    }  
  
    public Integer getId(){  
        return id;  
    }  
}
```

e. We can override Object(c) methods:

toString() equals() hashCode()

[because non-private, non-final,non-static]

f. Inheritance Rule:

Allowed ->Only Spring API is allowed (ie spring F/W classes and interface)

not allowed-> EJB, Servlet

g. Annotation Rule:

Only we can add Spring API Annotation and Inetegration API Annotations

(Hibernate with JPA, Restful webServices annotations);

ex:

@Component, @Service, @Repository.. etc.

STS Installation

=====

step 1:

go to official website

spring.io

link : - <https://spring.io/tools#eclipse>

step 2 download STS according your OS

step 3

Rigth click on ZIP file

click on "Extract Here"

step 4

Open extracted folder

Run : SpringToolsForEclipse

Final STEP STS installed successfully.

=====

session 6

=====

1. Spring Bean

Simple Java class that follow rules given by container.

2. Spring Configuration File(XML)

It gives object

details (name, data, links with other objects ... etc).

3. Test class=> Just find, container created or not? object is created or not?

=====

Spring core XML Tags

=====

<bean> - Object creation

<property> - calling set method

<constructor-arg> - call parameterized constructor

<value> - To provide data to Primitive Type.

<list> <set> <map> <props> -- To provide data to Collection type.

<ref/> -- To Link two objects

1. To create Object

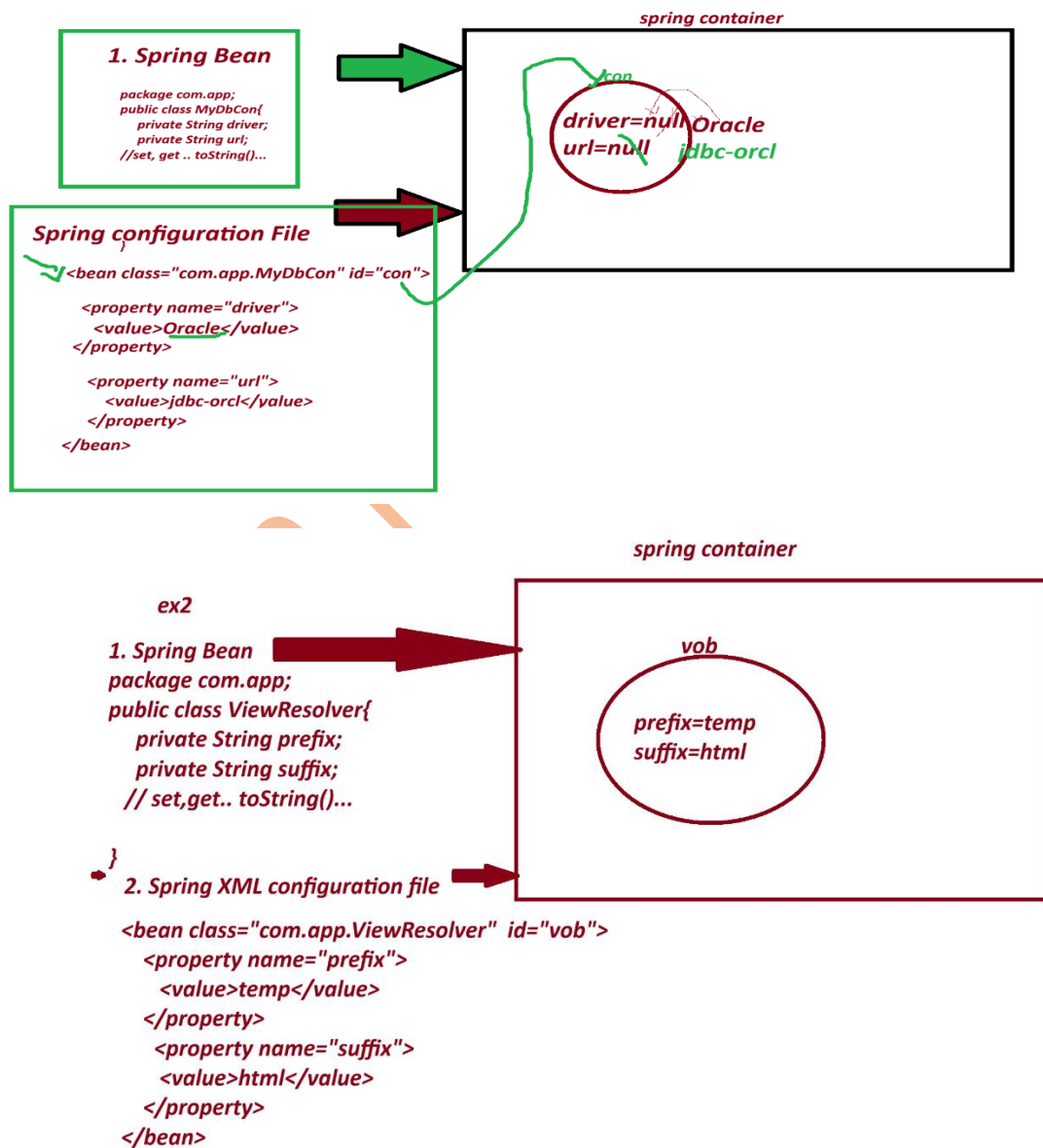
<bean id="objName" class="FullyQualifiedClassName"></bean>

2. To call set method

<property name="variableName"></property>

3. To provide data to Primitive Variables

<value>data</value>



1. Spring bean

package com.app;

```
public class PaymentService{  
  
    private int token;  
  
    private boolean active;  
  
    private String provider;  
  
    //set.. set toString..etc  
  
}
```

2. Spring XML configuration file.

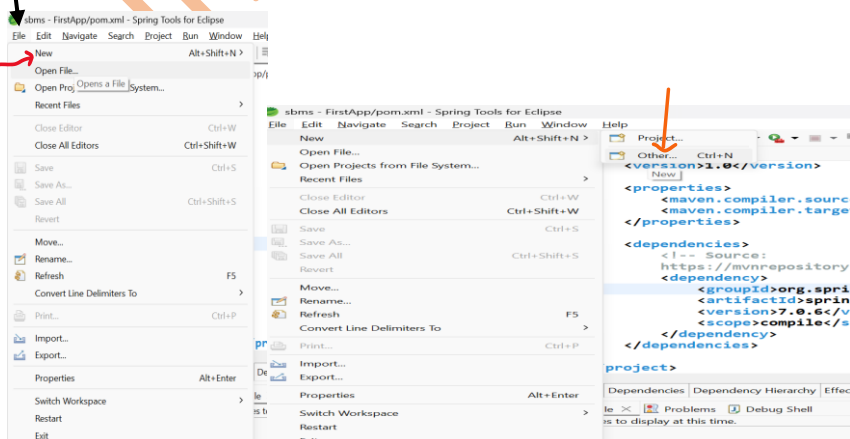
```
<bean class="com.app.PaymentService" id="psObj">  
  
    <property name="token">  
  
        <value>154678</value>  
  
    </property>  
  
    <property name="active">  
  
        <value>true</value>  
  
    </property>  
  
    <property name="provider">  
  
        <value>xyzBank</value>  
  
    </property>  
  
</bean>
```

=====

CREATE MAVEN PROJECT WITH BELOW STEP :-

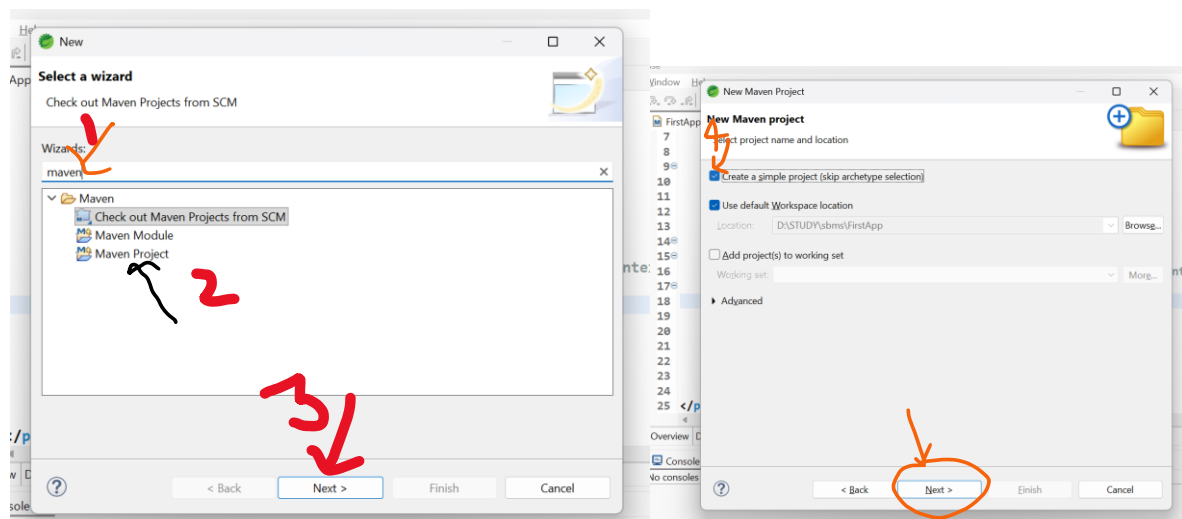
=====

1. Create Project



File-> New-> Other-> Type maven->choose maven project-> next->

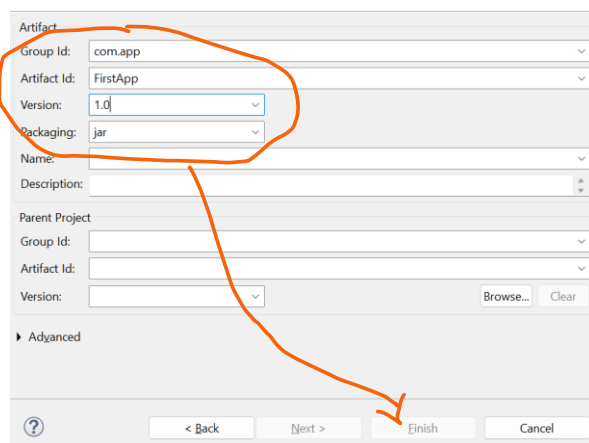
choose checkbox [v] create simple project -> next-> Enter details



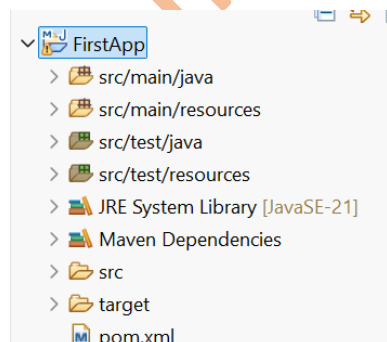
groupId : com.app

ArtifactId : FirstApp

version : 1.0



-> Finish



2. pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>FirstApp</artifactId>

<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

<dependencies>

    <!-- Source:
        https://mvnrepository.com/artifact/org.springframework/spring-context -->

    <dependency>

        <groupId>org.springframework</groupId>

        <artifactId>spring-context</artifactId>

        <version>7.0.6</version>

        <scope>compile</scope>

    </dependency>

</dependencies>

</project>

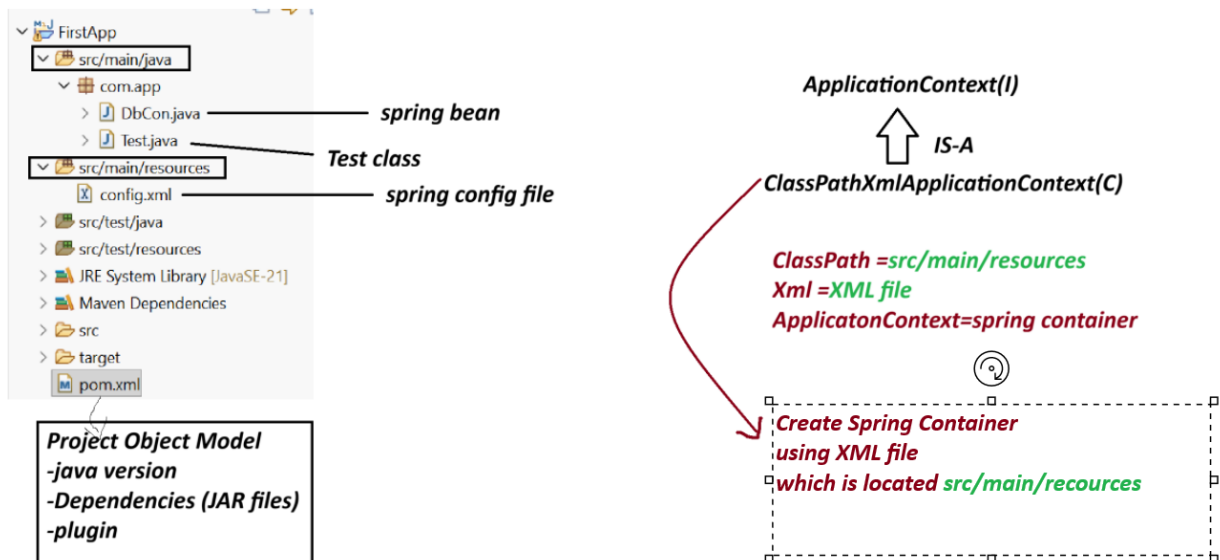
```

session 7

=====

Spring Core First Application 3 Files.

1. Spring bean (.java)
2. Spring XML configuration (config.xml)
3. Test class(.java)



Note:-

1. Spring container are two Types.

(a) Legacy Container : BeanFactory(I) support only XML

(b) New Container : ApplicationContext(i) support all XML/java/Annotation config.

(c) ApplicationContext(I) : it is a interface.

we are using one of its Impl class:

ClassPathXmlApplicationContext(C).

Create Maven Project with below Step:

File>new>other>type maven>choose maven project>next>choose checkbox> create Simple project>next>Enter details

GroupId - com.app

ArtficatId - SecondApp

version - 1.0

code

SecondApp

pom.xml

=====

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>
<artifactId>SecondApp</artifactId>
<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>

<dependencies>
<!-- Source:https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
<scope>compile</scope>
</dependency>
</dependencies>
</project>
```

DbCon.java

```
package com.app;

public class DbCon {
    private String driver;
    private String url;
```

```
//alt+shit+s

public DbCon() {
super();
System.out.println("Constructor called");
}

public String getDriver() {
return driver;
}

public void setDriver(String driver) {
this.driver = driver;
System.out.println("Driver method called");
}

public String getUrl() {
return url;
}

public void setUrl(String url) {
this.url = url;
System.out.println("Url method called");
}

@Override
public String toString() {
return "DbCon [driver=" + driver + ", url=" + url + "];"
}

}

```

=====

config.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```



```
xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean class="com.app.DbCon" id="mobj">
    <property name="driver">
        <value>Oracle Driver</value>
    </property>

    <property name="url">
        <value>JDBC-ORCL</value>
    </property>
</bean>
```

```
</beans>
```

```
=====
```

Test.java

```
-----
```

```
package com.app;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {
```

```
//ctrl+shit+O
```

```
public static void main(String[] args) {
```

```
    ApplicationContext ac=new ClassPathXmlApplicationContext("config.xml");
```

```
    Object ob = ac.getBean("mobj");
```

```
    System.out.println(ob);
```

```
}
```

```
}
```

output

Constructor called

Driver method called

Url method called

DbCon [driver=Oracle Driver, url=JDBC-ORCL]

session 8

=====

Spring Container (DI - Depedency Injection)

=>Depedency - Variables exist in class

3 type

1. Primitive Type (8+1) : byte, short, int, long, float, double, boolean, char, String

2. Collection Type (4) : Set, List, Map and Properties

3. Reference Type (x) : using a class/interface variable

=> Injection : provide Data

- Setter Injection

- constructor Injection

LookUpMethod Injetion

- Field Injection (Annotations)

=====

XML TAG

=====

<bean> - Object creation

<property> - calling set method

<constructor-arg> - call parameterized constructor

<value> - To provide data to Primitive Type.

<list> <set> <map> <props> -- To provide data to Collection type.

<ref/> -- To Link two objects

=====

Collection Configuration

=====

=> Spring container support working with collection configuration which are from java.util package

=====

Collection Name	Collection Type	XML Tag Name
=====	=====	=====
List	Interface	<list>
Set	Interface	<set>
Map	Interface	<map> and <entry>
Properties	Class	<props> and <prop>

List -> ArrayList

Set -> LinkedHashSet

Map -> LinkedHashMap

Properties --> NA

coding part collectionTypeS8

pom.xml

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.app</groupId>
<artifactId>collectionTypeS8</artifactId>
<version>1.0</version>
<properties>
<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>

```
</properties>
```

```
<dependencies><!-- Source:
```

```
https://mvnrepository.com/artifact/org.springframework/spring-context -->
```

```
<dependency>
```

```
<groupId>org.springframework</groupId>
```

```
<artifactId>spring-context</artifactId>
```

```
<version>7.0.5</version>
```

```
<scope>compile</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
</project>
```

```
-----  
Spring bean
```

```
MyDetails.java  
-----
```

```
package com.app;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.Properties;
```

```
import java.util.Set;
```

```
public class MyDetails {
```

```
//ctrl+shift+O
```

```
private List<String> data;
```

```
private Set<String> models;
```

```
private Map<Integer,String> design;
```

```
private Properties myinfo;
```

```
public MyDetails() {
```

```
super();
```

```
}

public List<String> getData() {
    return data;
}

public void setData(List<String> data) {
    this.data = data;
}

public Set<String> getModels() {
    return models;
}

public void setModels(Set<String> models) {
    this.models = models;
}

public Map<Integer, String> getDesign() {
    return design;
}

public void setDesign(Map<Integer, String> design) {
    this.design = design;
}

public Properties getMyinfo() {
    return myinfo;
}

public void setMyinfo(Properties myinfo) {
    this.myinfo = myinfo;
}

@Override
public String toString() {
```

```
return "MyDetails [data=" + data + ", models=" + models + ", design=" + design + ", myinfo=" + myinfo +
    "];"
}
}
```

xml config file

config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean class="com.app.MyDetails" id="myinfo">
```

```
<property name="data">
```

```
<list>
```

```
<value>A</value>
```

```
<value>B</value>
```

```
<value>C</value>
```

```
<value>A</value>
```

```
</list>
```

```
</property>
```

```
<property name="models">
```

```
<set>
```

```
<value>M1</value>
```

```
<value>M2</value>
```

```
<value>M3</value>
```

```
<value>M1</value>
```

```
</set>
```

```
</property>
```

<property name="design">

<map>

<entry>

<key>

<value>101</value>

</key>

<value>AB</value>

</entry>

<entry>

<key>

<value>102</value>

</key>

<value>Suraj</value>

</entry>

<entry>

<key>

<value>103</value>

</key>

<null></null>

</entry>

</map>

</property>

<property name="myinfo">

<props>

<prop key="A1">B1</prop>

<prop key="A2">B2</prop>

<prop key="A3">B3</prop>

<prop key="A5"></prop>

</props>

```
</property>
```

```
</bean>
```

```
</beans>
```

Test File

Test.java

```
package com.app;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        ApplicationContext ac=new ClassPathXmlApplicationContext("config.xml");
```

```
        //Object ob = ac.getBean("myinfo");
```

```
        MyDetails ob = ac.getBean("myinfo",MyDetails.class);
```

```
        System.out.println(ob);
```

```
        System.out.println(ob.getData().getClass().getName());
```

```
        System.out.println(ob.getModels().getClass().getName());
```

```
        System.out.println(ob.getDesign().getClass().getName());
```

```
        System.out.println(ob.getMyinfo().getClass().getName());
```

```
    }
```

```
}
```

console output

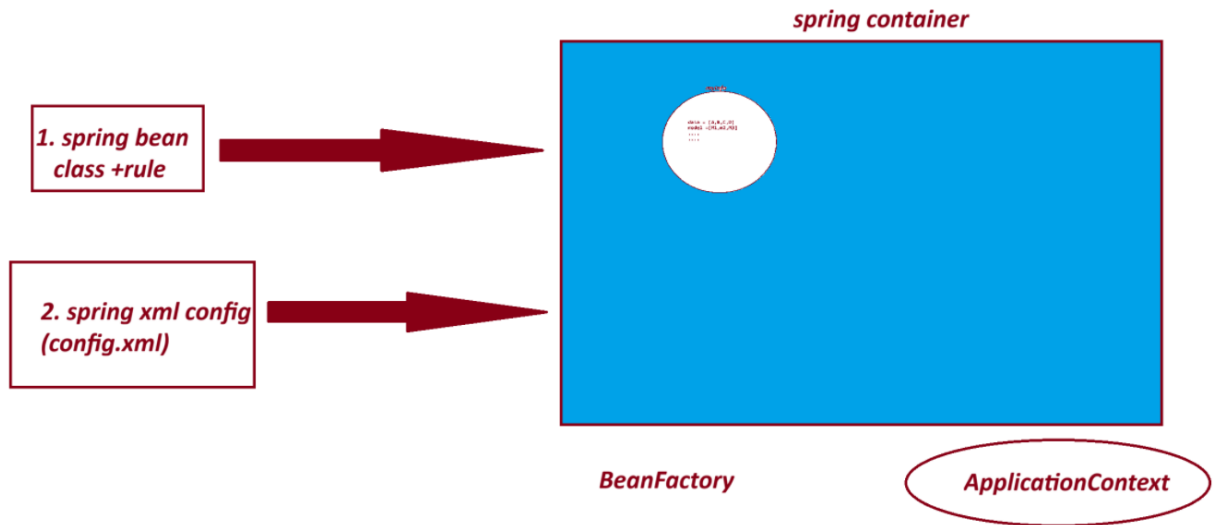
```
MyDetails [data=[A, B, C, A], models=[M1, M2, M3], design={101=AB, 102=Suraj, 103=null},  
myinfo={A1=B1, A2=B2, A3=B3, A5=}]
```

```
java.util.ArrayList
```

```
java.util.LinkedHashSet
```


java.util.LinkedHashMap

java.util.Properties



===

session 9

=====

3. Reference Type Dependency - Setter Injection.

=====

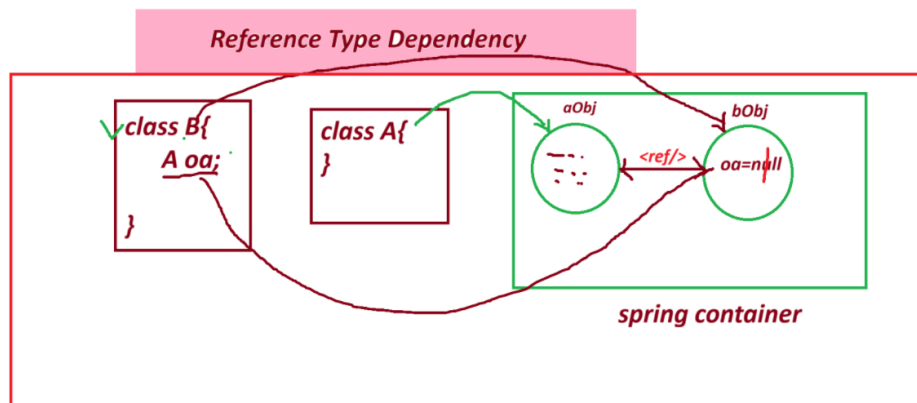
=> <ref/> : - tag is used to Link objects in case of HAS-A Relation.

Syntax:

<property name="variableName">

<ref bean="childObjName"/>

</property/>



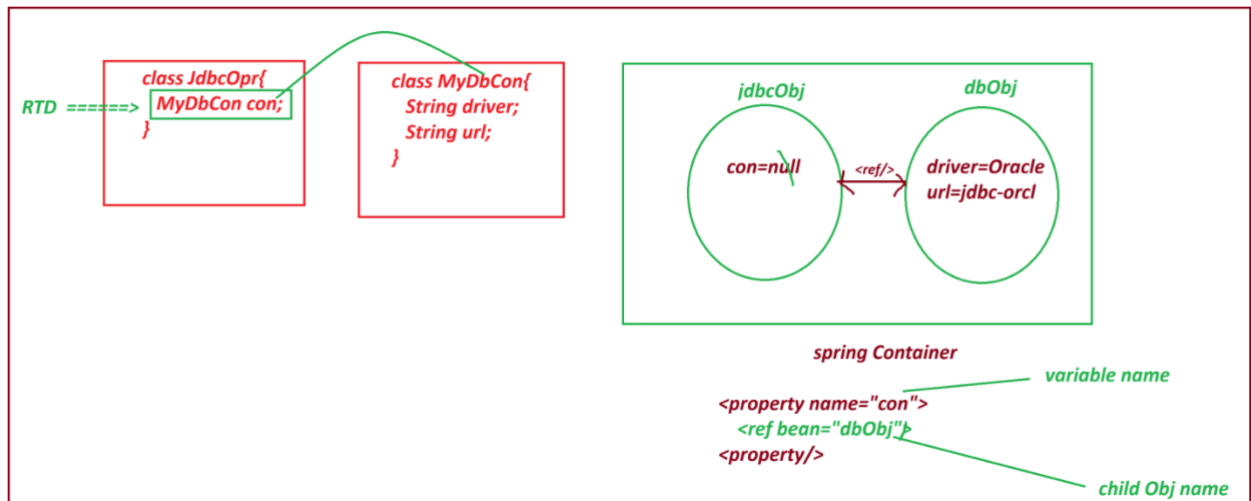
=====

- If we do not use <ref/> then ref variable holds default value null.

=> spring container creates object to both classes and link them by using <ref/> tag.

=> if child object nam (ref bean name) is not valid or not exist then spring container throws exception like
NoSuchBeanDefinitionException:

=====



project : ReferenceTypeS9

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.app</groupId>
<artifactId>ReferenceTypeS9</artifactId>
<version>1.0</version>
<properties>
<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>
<dependencies>
```

```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
<scope>compile</scope>
</dependency>
</dependencies>
```

```
</project>
```

```
-----
JdbcOpr.java
-----
```

```
package com.app;
```

```
public class JdbcOpr {
    private MyDbCon con; //RTD

    public JdbcOpr() {
        super();
        // TODO Auto-generated constructor stub
    }

    public MyDbCon getCon() {
        return con;
    }

    public void setCon(MyDbCon con) {
        this.con = con;
    }
}
```

```

@Override

public String toString() {
return "JdbcOpr [con=" + con + "];
}

```

```

}

```

```

=====

```

MyDbCon.java

```

-----

```

```

package com.app;

```

```

public class MyDbCon {

```

```

    private String driver;

```

```

    private String url;

```

```

    public MyDbCon() {

```

```

        super();

```

```

        // TODO Auto-generated constructor stub

```

```

    }

```

```

    public String getDriver() {

```

```

        return driver;

```

```

    }

```

```

    public void setDriver(String driver) {

```

```

        this.driver = driver;

```

```

    }

```

```

    public String getUrl() {

```

```

        return url;

```

```

    }

```

```

    public void setUrl(String url) {
this.url = url;
    }

    @Override
    public String toString() {
return "MyDbCon [driver=" + driver + ", url=" + url + "];"
    }

    //alt+shift+s

}

```

config.xml

```

-----
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean class="com.app.MyDbCon" id="dbObj" >
        <property name="driver">
            <value>OracleDriver</value>
        </property>
        <property name="url">
            <value>JDBC-ORCL</value>
        </property>
    </bean>

    <bean class="com.app.JdbcOpr" id="jdbcObj">
        <property name="con">
            <ref bean="dbObj"/>

```

</property>

</bean>

</beans>

Test.java

package com.app;

import org.springframework.context.ApplicationContext;

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

public static void main(String[] args) {

ApplicationContext ac = new ClassPathXmlApplicationContext("config.xml");

JdbcOpr jdbc = ac.getBean("jdbcObj", JdbcOpr.class);

System.out.println(jdbc);

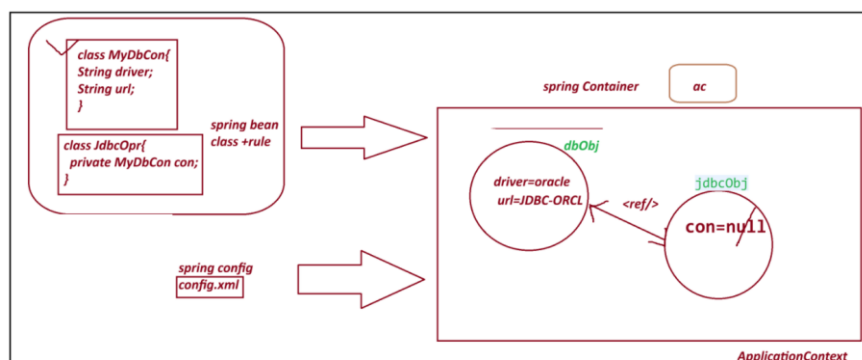
}

}

output

JdbcOpr [con=MyDbCon [driver=OracleDriver, url=JDBC-ORCL]]

=====



SI - Setter Injection :

Spring container creates object to given class using default constructor and provide data using setter method.

Employee emp=new Employee(); =====> <bean id="emp" class="Employee"/>

emp.setEmpId(10); =====> <property name="empId" value="10"/>

=====

*** Constructor Injection CI *****

=====

=> Spring container creating object and providing data using Parameterized constructor.

Here , we use <constructor-arg> tag to indicate.

Employee emp=new Employee(10, "A"); =====>

<bean>

<constructor-arg>

<value>10</value>

</constructor-arg>

<constructor-arg>

<value>A</value>

</constructor-arg>

</bean>

Note:

A. If a class has 3 param constructor then we need to pass <constructor-arg> 3 times in xml config file.

B. Data must be passed in order using <constructor-arg> as per constructor defined in class.

=====

code for CI (constructor Injection)

pom.xml

```

-----
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>ConstructorInjectionS9</artifactId>

<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>7.0.5</version>

<scope>compile</scope>

</dependency>

</dependencies>

</project>

```

```

-----
Product.java

```

```

package com.app;

```

```

import java.util.List;

```

```

public class Product {

```

```

private String pcode;

```



```

private String model;

private List<String> data;

private Vendor vob;


public Product(String pcode, String model, List<String> data, Vendor vob) {
    super();
    this.pcode = pcode;
    this.model = model;
    this.data = data;
    this.vob = vob;
}

@Override
public String toString() {
    return "Product [pcode=" + pcode + ", model=" + model + ", data=" + data + ", vob=" + vob + "];"
}

}

```

Test.java

```

package com.app;

public class Vendor {

    private String vcode;
    private String vaddr;


    public Vendor(String vcode, String vaddr) {
        super();
        this.vcode = vcode;
        this.vaddr = vaddr;
    }
}

```

@Override

```
public String toString() {  
    return "Vendor [vcode=" + vcode + ", vaddr=" + vaddr + "];"  
}  
  
}
```

config.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean class="com.app.Vendor" id="v1">
```

```
    <constructor-arg>
```

```
        <value>v1109</value>
```

```
    </constructor-arg>
```

```
    <constructor-arg>
```

```
        <value>IND</value>
```

```
    </constructor-arg>
```

```
</bean>
```

```
<bean class="com.app.Product" id="pobj">
```

```
    <constructor-arg>
```

```
        <value>P01</value>
```

```
    </constructor-arg>
```

```
    <constructor-arg>
```

<value>HP-23444</value>

</constructor-arg>

<constructor-arg>

<list>

<value>AA</value>

<value>BB</value>

</list>

</constructor-arg>

<constructor-arg>

<ref bean="v1"/>

</constructor-arg>

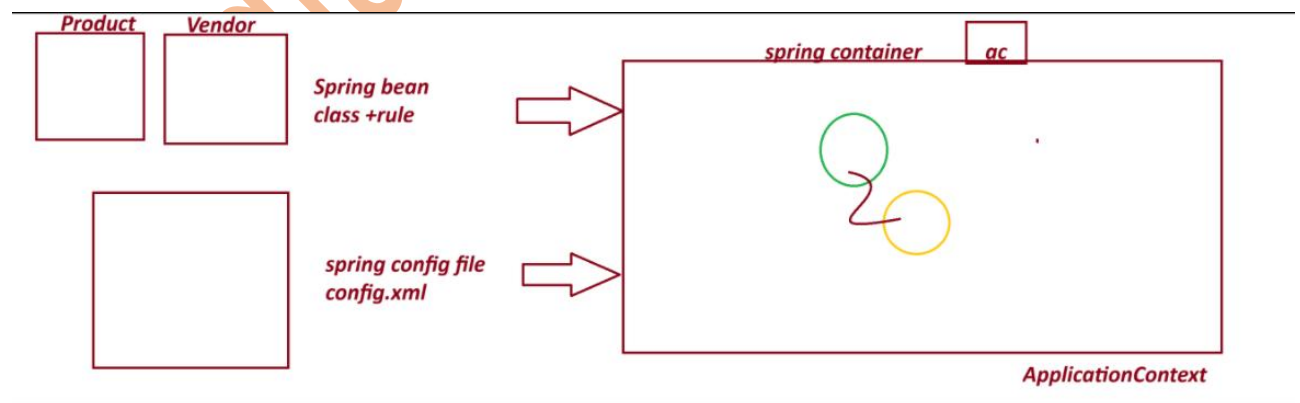
</bean>

</beans>

output

=====

Product [pcode=P01, model=HP-23444, data=[AA, BB], vob=Vendor [vcode=v1109, vaddr=IND]]



=====

session 10

=====

Spring Annotation Configuration

This is Easy to Configure

Faster in execution compare to xml.

=====

1. Stereotype Annotation:

These annotation informs Spring Container to create Object.

They are 5 Annotation given by Spring Framework.

@Component : Create object inside container.

@Repository : Creates object + Database operations

@Service : Creates object + Transaction/logic/calculation..etc

@Controller : Creates object + Http Protocol (Web App)

@RestController : Creates object + Http Protocol (webservices)

@ComponentScan

@ComponentScans

2. Data Annotation/ Basic Annotation

@Value Annotation is used to provide data (field inject) to variable.

It has 3 usages:

1. Hard coding direct value to a variable.
2. Reading Data from Properties/YAML Files
3. - SpEL (Spring Expression language)

@Autowire

@Qualifier

@Primary

=====

ExcelExportAnnotationConfigS10

code

=====

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>ExcelExportAnnotationConfigS10</artifactId>

<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>7.0.5</version>

<scope>compile</scope>

</dependency>

</dependencies>

</project>
```

Spring bean + annotation configuration

ExcelExport.java

```
package com.app;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```

import org.springframework.stereotype.Component;

@Component("excel")

public class ExcelExport {

    @Value("TEST")

    private String exportName;

    @Value(".csv")

    private String fileExt;


    @Override

    public String toString() {

        return "ExcelExport [exportName=" + exportName + ", fileExt=" + fileExt + "];

    }

}

```

MyAppConfig.java

```

package com.app;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.ComponentScans;

```

```

@ComponentScan("com.app")

public class MyAppConfig {

}

```

Test.java

```

package com.app;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

```

```
import org.springframework.context.ApplicationContext;
```

```
public class Test {
```

```
public static void main(String[] args) {
```

```
AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(MyAppConfig.class);
```

```
    // Find classes from a package(basePackage)
```

```
    //ac.scan("com.app");
```

```
    // object creation
```

```
    //ac.refresh();
```

```
    ExcelExport ee = ac.getBean("excel",ExcelExport.class);
```

```
    System.out.println(ee);
```

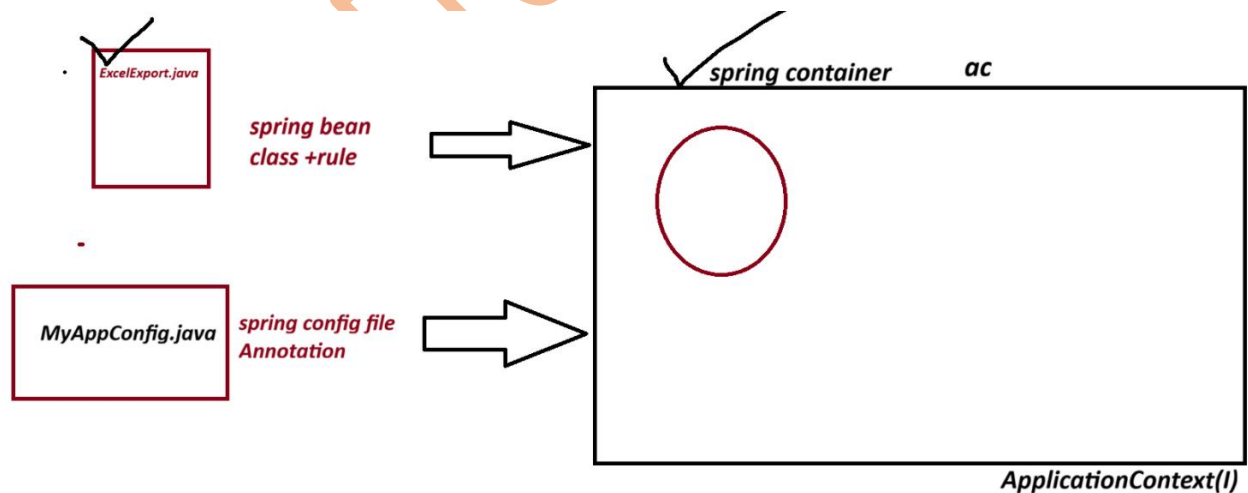
```
}
```

```
}
```

output

ExcelExport [exportName=TEST, fileExt=.csv]

Diagram



=====

session 11

=====

Spring Annotation Configuration:

1. @Component : create Object to given class

2. @Value : To provide data (Field Injection) to variable

3. @ComponentScan : To find classes using basepackage.

=====Properties File=====

_____.properties

=> This is also called as key-val pair input as Text format(String)

=> This file is used to provide setup data to application.

=> Here, we can read data from properties file into variable using @Value("\${key}")

=> Programmer has to define and load Properties file.

Use @PropertySource annotation to load file, into spring container.

=> container creates Environment() Object (internally used StandardEnvironment(c)).

=> we can read data using @Value("\${keyName}")

=> if we define duplicate key with different value (not recommended then last combination is taken into Container)

=====

myapp.properties

#Symbol allowed for comment '#'

#key=value

#Symbol allowed to write keyName (dot(.), dash(-), underscore(_))

my.driver=oracle

my.url=oracle-jdbc

my.username=system

my.password=root

my.port=3306

my.active=false

=====

my.port=4444 will be printed

=====

=> For boolean datatype valid input are : false and true

(case-insensitive, False, FALSE, True, TRUE etc)

=====

Project Name: database-connectionS11

Example

code

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>database-connectionS11</artifactId>

<version>1.0</version>

<properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>7.0.5</version>

<scope>compile</scope>

</dependency>

</dependencies>

</project>
```

Test.java

```
package com.app;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```

public class Test {

    public static void main(String[] args) {
        ApplicationContext ac=new AnnotationConfigApplicationContext(MyAppConfig.class);
        DatabaseCon obj = ac.getBean("databaseCon",DatabaseCon.class);
        System.out.println(obj);
    }

}

```

DatabaseCon.java

```
package com.app;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
```

```
@Component
```

```
public class DatabaseCon {
```

```
@Value("${my.db}")
```

```
private String driver;
```

```
@Value("${my.url}")
```

```
private String url;
```

```
@Value("${my.username}")
```

```
private String username;
```

```
@Value("${my.password}")
```

```
private String password;
```

```
@Value("${my.port}")
```

```
private int port;
```

```
@Value("${my.enable}")
```

```
private boolean active;
```

```
@Override
```

```
public String toString() {
```

```
return "DatabaseCon [driver=" + driver + ", url=" + url + ", username=" + username + ", password=" +  
password
```

```
+ ", port=" + port + ", active=" + active + "];
```

```
}
```

```
}
```

```
-----  
MyAppConfig.java
```

```
package com.app;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
import org.springframework.context.annotation.PropertySource;
```

```
@ComponentScan("com.app")
```

```
@PropertySource("classpath:myapp.properties")
```

```
public class MyAppConfig {
```

```
}
```

```
-----  
myapp.properties
```

```
# comment line
```

```
# symbol allowed to write keyName (dot . dash - underscore _)
```

```
#key=value
```

```
my.db=oracle
```

my.url=oracle-jdbc

my.username=indiafreeinternship

my.password=root

my.port=3306

#my.port=3345

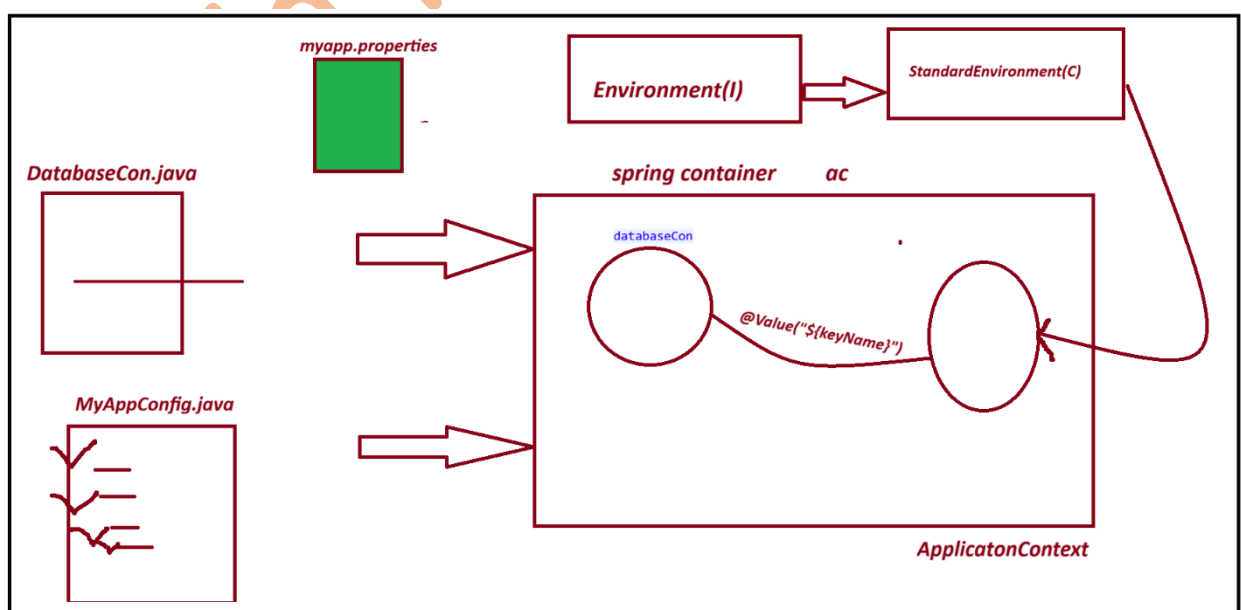
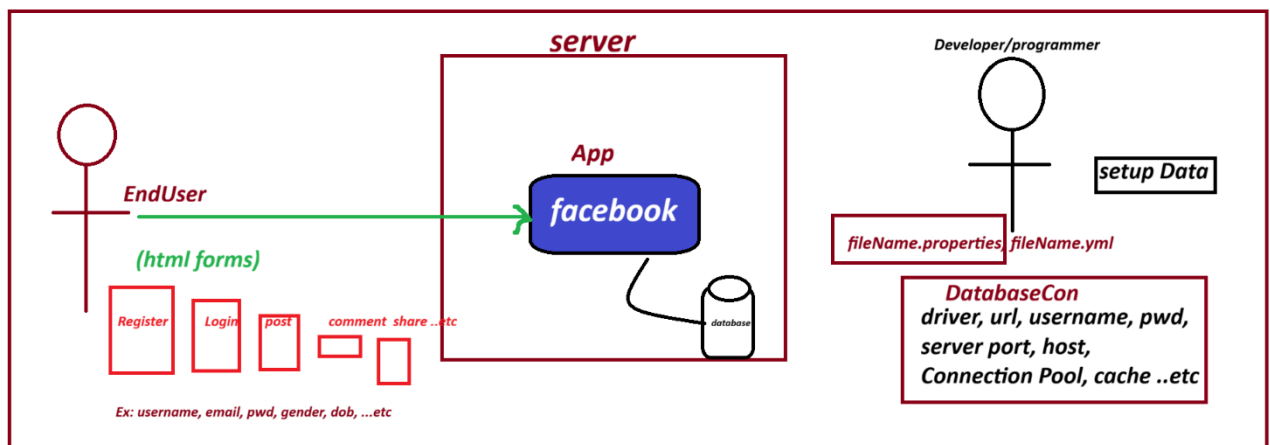
#my.port=4444

my.enable=true

output

DatabaseCon [driver=oracle, url=oracle-jdbc, username=indiafreeinternship, password=root, port=4444, active=true]

Diagram



session 12

=====

Spring Annotation configuration

@Component Annotation : To create Object

@Value : Direct value to a variable / Read data from properties

@ComponentScan : back-Package to scan classes

@PropertySource : To Load Properties File

Autowiring

@Autowired Annotation

=> It is a process of Linking objects based on Association Mapping (HAS-A)

=> @Autowired Annotation is used. So, that container can link them.

=> Autowired Annotation Injects Dependecny Beans into Dependent bean.

=> In Case of XML Configuration <ref/> tag is used to link objects.

=====

1. @Autowired will search for child class object.

2. If one child object is found then it is injected.

3. If Zero child object are found then throw exception NoSuchBeanDefinitionException, message like:

At least 1 bean required

=====

-----code-----

project name : AutowiredAnnoS12

=====

pom.xml

<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.app</groupId>

<artifactId>AutowiredAnnoS12</artifactId>

```

<version>1.0</version>

<properties>

<maven.compiler.source>8</maven.compiler.source>

<maven.compiler.target>8</maven.compiler.target>

</properties>

<dependencies>

<!-- Source:
https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
<scope>compile</scope>
</dependency>
</dependencies>

</project>

```

2. Spring bean

EmployeeDao.java

```

package com.app;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component("edao")
@Component("edao")
public class EmployeeDao {

```

```
@Value("ORACLE")
```

```
private String dbName;
```

```
@Override
```

```
public String toString() {
```

```
return "EmployeeDao [dbName=" + dbName + "];
```

```
}
```

```
}
```

```
-----
```

3. spring bean

EmployeeService.java

```
-----
```

```
package com.app;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Component;
```

```
//ctrl+shift+O
```

```
@Component("esobj")
```

```
public class EmployeeService {
```

```
//@Autowired(required=true)
```

```
@Autowired(required=false)
```

```
private EmployeeDao dao; // HAS-A
```

```
@Override
```

```
public String toString() {
```

```
return "EmployeeService [dao=" + dao + "];
```

```
}
```

```
}
```

```
-----  
AppConfig.java  
-----
```

```
package com.app;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
@ComponentScan("com.app")
```

```
public class AppConfig {
```

```
}
```

```
-----  
Test.java  
-----
```

```
package com.app;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class Test {
```

```
public static void main(String[] args) {
```

```
ApplicationContext ac=new AnnotationConfigApplicationContext(AppConfig.class);
```

```
EmployeeService ob = ac.getBean("esobj",EmployeeService.class);
```

```
System.out.println(ob);
```

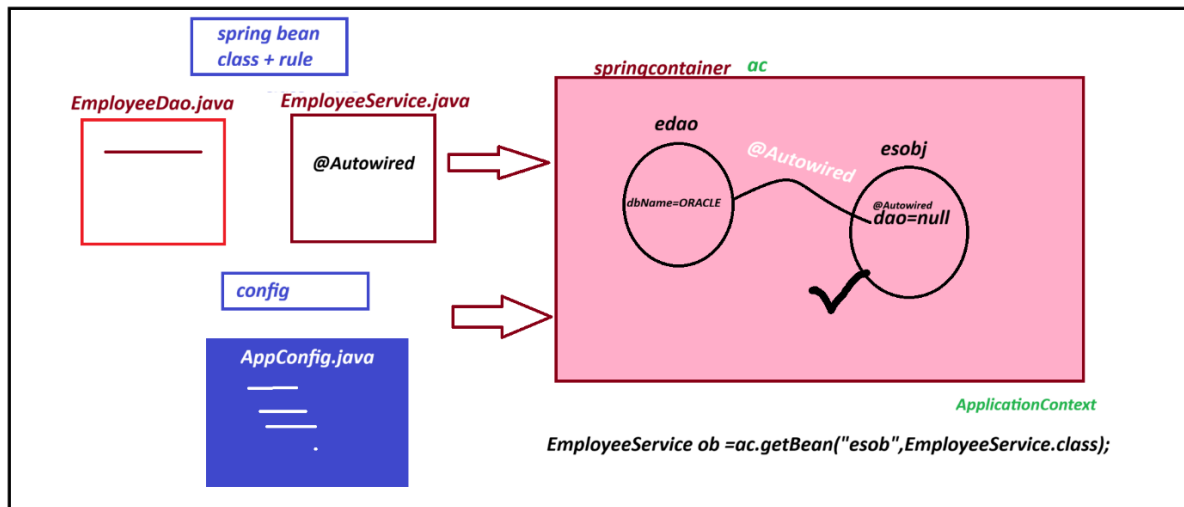
```
}
```

```
}
```

Output

```
EmployeeService [dao=EmployeeDao [dbName=ORACLE]]
```


Diagram



Sessino 13

Spring Bean -LifeCycle Methods

LifeCycle Methods : If a method is called by container/runtime in between Object creation and destroy.

Spring F/W : 2 LifeCycle Methods (they are optional)

1. init-method : called after creating object
2. destroy-method : called before Destroying object

File(data) -----> open file-----> Read data-----variable-> close File

We can implement LifeCycle Methods in 3 way

1. xml cofiguration

```
<bean ..... init-method="" destroy-method="">
```

=>we can define methods with any name, using below syntax:

```
public void <method_name>(){
```

```
//logic....
```

}

2. spring Lifecycle Interface:

=> Spring f/w has given 2 interfaces to implement lifecycle methods.

They are:

A. InitializingBean(I)

afterPropertiesSet()

B. DisposableBean(I)

Destroy()

=> Your Spring bean can implement above interface and override methods.

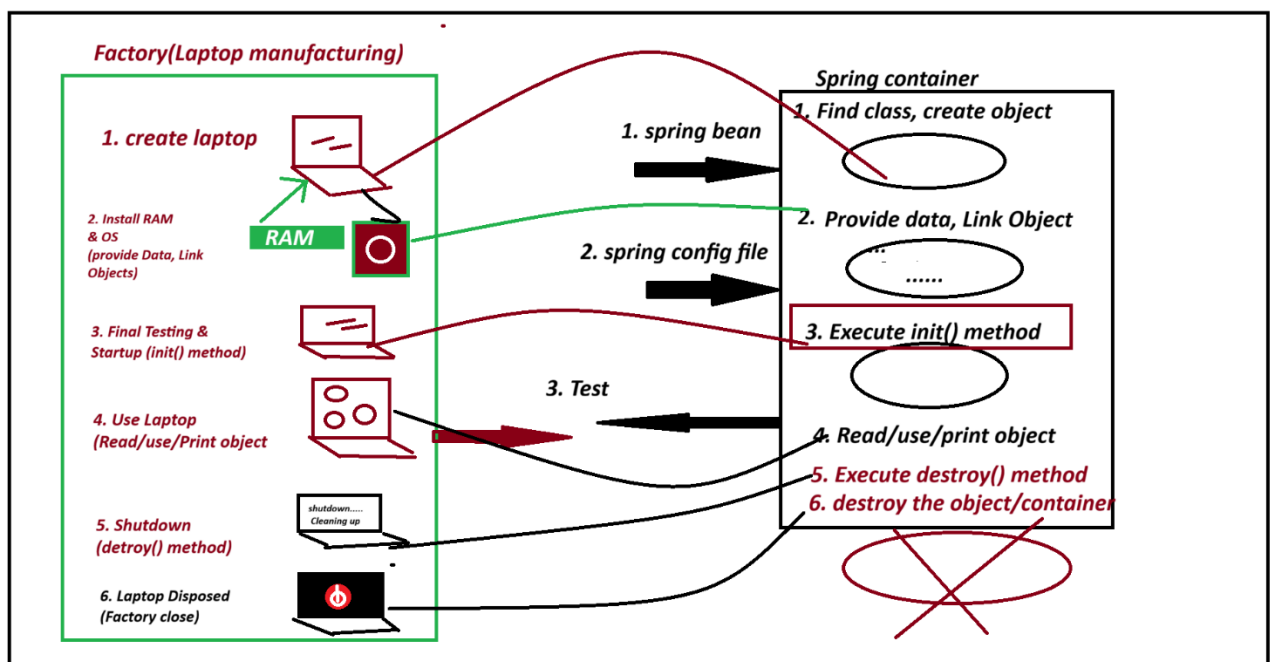
They are called by spring container.

3. JSR-Annotation : These are generic annotation Given by Sun/oracle .

a. @PostConstruct

b. @PreDestroy

diagram



=====code=====

1. xml cofiguration

```
<bean ..... init-method="" destroy-method="">
```

=>we can define methods with any name, using below syntax:

```
    public void <method_name>(){  
//logic....  
}
```

=====code=====

project :- SpringBeanLifeCycleXMLS13

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-  
4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.app</groupId>
```

```
    <artifactId>SpringBeanLifeCycleXMLS13</artifactId>
```

```
    <version>0.0.1-SNAPSHOT</version>
```

```
    <properties>
```

```
    <maven.compiler.source>8</maven.compiler.source>
```

```
    <maven.compiler.target>8</maven.compiler.target>
```

```
    </properties>
```

```
    <dependencies>
```

```
    <!-- Source:
```

```
    https://mvnrepository.com/artifact/org.springframework/spring-context -->
```

```
    <dependency>
```

```
    <groupId>org.springframework</groupId>
```

```
    <artifactId>spring-context</artifactId>
```

```
    <version>7.0.5</version>
```

```
    <scope>compile</scope>
```

```
    </dependency>
```

```
</dependencies>
```

```
</project>
```

```
-----
```

```
spring bean
```

```
ExcelExport.java
```

```
package com.app;
```

```
public class ExcelExport {
```

```
private String fileName;
```

```
private String fileExt;
```

```
public ExcelExport() {
```

```
super();
```

```
System.out.println("from constructor");
```

```
}
```

```
public String getFileName() {
```

```
return fileName;
```

```
}
```

```
public void setFileName(String fileName) {
```

```
this.fileName = fileName;
```

```
System.out.println("from setter method");
```

```
}
```

```
public String getFileExt() {
```

```
return fileExt;
```

```
}
```

```
public void setFileExt(String fileExt) {
```

```
this.fileExt = fileExt;
```

```
}
```

```
public void setUp(){  
    System.out.println("3.FROM INIT METHOD");  
}
```

```
public void clear() {  
    System.out.println("5.FROM DESTORY METHOD");  
}
```

```
@Override  
public String toString() {  
    return "ExcelExport [fileName=" + fileName + ", fileExt=" + fileExt + "];"  
}  
  
}
```

2. config.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean class="com.app.ExcelExport" id="exObj" init-method="setUp" destroy-method="clear">  
        <property name="fileName" value="ABC"/>  
        <property name="fileExt" value="CSV"/>  
    </bean>  
  
</beans>
```

3. Test.java

```
package com.app;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {
```

```
    @SuppressWarnings("resource")
```

```
    public static void main(String[] args) {
```

```
        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("config.xml");
```

```
        ExcelExport ob = ac.getBean("exObj",ExcelExport.class);
```

```
        System.out.println("FROM MAIN");
```

```
        System.out.println(ob);
```

```
        ac.registerShutdownHook();
```

```
        //ac.close();
```

```
    }
```

```
}
```

output

from constructor

from setter method

3.FROM INIT METHOD

FROM MAIN

ExcelExport [fileName=ABC, fileExt=CSV]

5.FROM DESTORY METHOD

=====

2. spring Lifecycle Interface:

=> Spring f/w has given 2 interfaces to implement lifecycle methods.

They are:

A. InitializingBean(I)

afterPropertiesSet()

B. DisposableBean(I)

Destroy()

=> Your Spring bean can implement above interface and override methods.

They are called by spring container.

=====code=====

project Name :SpringBeanLifeCycleInterfaceS13

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
```

```
    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.app</groupId>
```

```
    <artifactId>SpringBeanLifeCycleXMLS13</artifactId>
```

```
    <version>0.0.1-SNAPSHOT</version>
```

```
    <properties>
```

```
        <maven.compiler.source>8</maven.compiler.source>
```

```
        <maven.compiler.target>8</maven.compiler.target>
```

```
    </properties>
```

```
    <dependencies>
```

```
        <!-- Source:
```

```
        https://mvnrepository.com/artifact/org.springframework/spring-context -->
```

```
    <dependency>
```

```
        <groupId>org.springframework</groupId>
```

```
        <artifactId>spring-context</artifactId>
```

```
        <version>7.0.5</version>
```

```
        <scope>compile</scope>
```

```
    </dependency>
```

```
    </dependencies>
```

```
</project>
```

ExcelExport.java

```
package com.app;
```

```
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;

public class ExcelExport implements InitializingBean, DisposableBean{

    private String fileName;
    private String fileExt;

    public ExcelExport() {
        super();
        System.out.println("from constructor");
    }

    public String getFileName() {
        return fileName;
    }

    public void setFileName(String fileName) {
        this.fileName = fileName;
        System.out.println("from setter method");
    }

    public String getFileExt() {
        return fileExt;
    }

    public void setFileExt(String fileExt) {
        this.fileExt = fileExt;
    }
}
```



```

@Override

public String toString() {

return "ExcelExport [fileName=" + fileName + ", fileExt=" + fileExt + "];"

}

```

```

@Override

public void afterPropertiesSet() throws Exception {

System.out.println("FROM INIT METHOD AFTER PROPERTIESSET()");

}

```

```

@Override

public void destroy() throws Exception {

System.out.println("from destroy method destroy()");

}

```

```

}

```

```

=====

```

config.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean class="com.app.ExcelExport" id="exObj">

        <property name="fileName" value="ABC"/>

        <property name="fileExt" value="CSV"/>

    </bean>

</beans>

```

Test.java

package com.app;

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

@SuppressWarnings("resource")

public static void main(String[] args) {

ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("config.xml");

ExcelExport ob = ac.getBean("exObj",ExcelExport.class);

System.out.println("FROM MAIN");

System.out.println(ob);

ac.registerShutdownHook();

//ac.close();

}

}

output

from constructor

from setter method

FROM INIT METHOD AFTER PROPERTIESSET()

FROM MAIN

ExcelExport [fileName=ABC, fileExt=CSV]

from destroy method destroy()

=====

3. JSR-Annotation : These are generic annotation Given by Sun/oracle .

a. @PostConstruct

b. @PreDestroy

-----code-----

```
<dependency>
<groupId>jakarta.annotation</groupId>
<artifactId>jakarta.annotation-api</artifactId>
<version>2.0.0</version>
</dependency>
```

project name : SpringBeanLifeCycleAnnotationS13

code

ExcelExport.java

package com.app;

import org.springframework.beans.factory.annotation.Value;

import org.springframework.stereotype.Component;

import jakarta.annotation.PostConstruct;

import jakarta.annotation.PreDestroy;

@Component("exObj")

public class ExcelExport {

@Value("SAMPLE")

private String fileName;

@Value(".csv")

private String fileExt;

public ExcelExport() {

super();

```
System.out.println("from constructor");  
}
```

@Override

```
public String toString() {return "ExcelExport [fileName=" + fileName + ", fileExt=" + fileExt + "];"  
}
```

@PostConstruct

```
public void setUpA() {  
System.out.println("FROM INIT METHOD");  
}
```

@PreDestroy

```
public void clearB() {  
System.out.println("FROM DESTROY METHOD");  
}
```

```
}
```

Test.java

```
package com.app;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class Test {
```

```
@SuppressWarnings("resource")
```

```
public static void main(String[] args) {
```

```
AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
ExcelExport ob = ac.getBean("exObj",ExcelExport.class);
```

```
System.out.println("FROM MAIN");
```

```
System.out.println(ob);
```

```
ac.registerShutdownHook();  
  
//ac.close();  
  
}
```

```
}
```

```
-----
```

AppConfig.java

```
package com.app;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
@ComponentScan("com.app")
```

```
public class AppConfig {
```

```
}
```

```
-----
```

pom.xml

```
-----
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-  
4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.app</groupId>
```

```
<artifactId>SpringBeanLifeCycleXMLS13</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
```

```
<properties>
```

```
<maven.compiler.source>8</maven.compiler.source>
```

```
<maven.compiler.target>8</maven.compiler.target>
```

```
</properties>
```

```
<dependencies>
```

```
<!-- Source:
```

<https://mvnrepository.com/artifact/org.springframework/spring-context> -->

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>7.0.5</version>

<scope>compile</scope>

</dependency>

<dependency>

<groupId>jakarta.annotation</groupId>

<artifactId>jakarta.annotation-api</artifactId>

<version>2.0.0</version>

</dependency>

</dependencies>

</project>

output

from constructor

FROM INIT METHOD

FROM MAIN

ExcelExport [fileName=SAMPLE, fileExt=.csv]

FROM DESTROY METHOD

=====

session 14

=====

Spring - Java Configuration

=====

XML Configuration -- <bean> <property> <value> .. etc

Annotation configuration -- @Component, @Value, @Autowired .etc

java configuration -@Configuration @Bean

=====

java configuration -@Configuration @Bean

1. Define one public class with any name (ex MyApp.java)
2. Apply @Configuration another over class
3. For one object = define one method

@Bean

```
public PdfExport pObj(){  
    PdfExport p = new PdfExport();  
    p.setFileName("sbms");  
    p.setFileAuth("IndiaFreeInternship");  
    return p;  
  
}
```

4. Add @Bean over method

1. Consider given class (spring bean) is a pre-defined class.

can we use Annotation config?

=> No. We can't Annotation configuration.

we can use annotation configuration we have source code only.

(_____.java) file

2. Can we xml/java configuration to define one bean for pre-defined class?

yes Both support any type of class of configuration (pre-defined and user-defined)

3. what is the mean of @Configuration and @Bean?

@Configuration says "i am java config file"

@Bean - please create one object inside spring container

4. what is the diff b/w @Component and @Bean?

@Component - used for programmer defined

@Bean - Mainly for pre-defined class (support both pre-defined and user defined)

5. why we use Annotation configuration mostly

execution fast (short code)

in java configuration (lengthy code and slow execution)

=====

example code

PdfExportJavaConfigS14

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
```

```
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.app</groupId>
```

```
  <artifactId>PdfExportJavaConfigS14</artifactId>
```

```
  <version>1.0</version>
```

```
  <properties>
```

```
    <maven.compiler.source>8</maven.compiler.source>
```

```
    <maven.compiler.target>8</maven.compiler.target>
```

```
  </properties>
```

```
  <dependencies>
```

```
    <!-- Source:
```

```
    https://mvnrepository.com/artifact/org.springframework/spring-context -->
```

```
  <dependency>
```

```
    <groupId>org.springframework</groupId>
```

```
    <artifactId>spring-context</artifactId>
```

```
    <version>7.0.5</version>
```

```
    <scope>compile</scope>
```

```
  </dependency>
```

```
  <dependency>
```

```
    <groupId>jakarta.annotation</groupId>
```



```
<artifactId>jakarta.annotation-api</artifactId>
```

```
<version>2.0.0</version>
```

```
</dependency>
```

```
<!-- Source: https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
```

```
<dependency>
```

```
  <groupId>org.springframework</groupId>
```

```
  <artifactId>spring-jdbc</artifactId>
```

```
  <version>7.0.7</version>
```

```
  <scope>compile</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
</project>
```

```
-----
```

```
Test.java
```

```
package com.app.test;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
import com.app.bean.PdfExport;
```

```
import com.app.config.AppConfig;
```

```
public class Test {
```

```
  @SuppressWarnings("resource")
```

```
  public static void main(String[] args) {
```

```
    AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
    PdfExport ob = ac.getBean("pObj", PdfExport.class);
```

```
    System.out.println(ob);
```

```
  }
```

```
}
```

```
-----
```

PdfExport.java

```
package com.app.bean;
```

```
public class PdfExport {
```

```
    private String fileName;
```

```
    private String fileAuth;
```

```
    public String getFileName() {
```

```
        return fileName;
```

```
    }
```

```
    public void setFileName(String fileName) {
```

```
        this.fileName = fileName;
```

```
    }
```

```
    public String getFileAuth() {
```

```
        return fileAuth;
```

```
    }
```

```
    public void setFileAuth(String fileAuth) {
```

```
        this.fileAuth = fileAuth;
```

```
    }
```

```
    @Override
```

```
    public String toString() {
```

```
        return "PdfExport [fileName=" + fileName + ", fileAuth=" + fileAuth + "];"
```

```
    }
```

```
}
```

```
-----
```

AppConfig.java

```
package com.app.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import com.app.bean.PdfExport;
```

```
@Configuration
```

```
public class AppConfig {
```

```
// user defined
```

```
@Bean
```

```
public PdfExport pObj(){
```

```
    PdfExport p = new PdfExport();
```

```
    p.setFileName("sbms");
```

```
    p.setFileAuth("IndiaFreeInternship");
```

```
    return p;
```

```
}
```

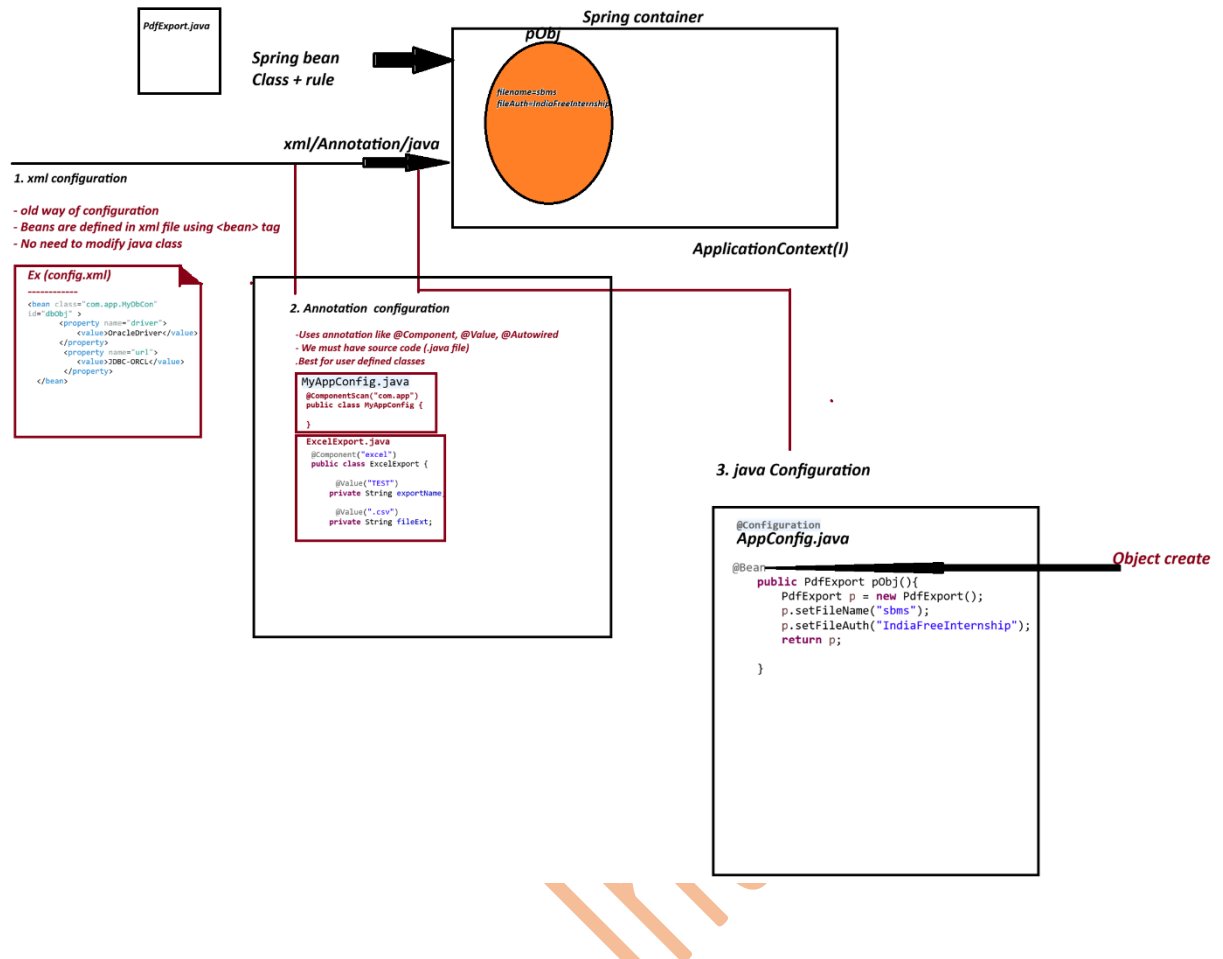
```
}
```

```
-----
output
-----
```

```
PdfExport [fileName=sbms, fileAuth=IndiaFreeInternship]
```

```
Diagram
```

```
=====
```



Point	xml configuration	Annotation Configuration	Java Configuration
How to define Bean	<bean> tag in XML file	@Component, @Value, @Autowired	@Configuration class with @Bean tag
Need of Source Code	No	Yes(must have .java file)	No (Work for both)
Supports	Both pre-defined and user-defined classes	Only User-defined classes	Both pre-defined and user-defined classes
Multiple beans	Yes	Yes	Yes(easier)
Readability	Medium	High	High
Moder Approach	No	Yes(most used)	Yes

PRE-DEFINED CLASS

classes which are already available in libraries or framework.

we don't have source code. (.java)

ex:

JdbcTemplate
 DriverManagerDataSource
 RestTemplate
 ArrayList(java API)

We Cannot apply @Component
 (Cannot modify source code)

USER-DEFINED CLASS

Classes which are created by Programmer
 We have source code (.java)

Ex

Student.java
 Employee.java
 Product.java
 PdfExport.java

We can apply @Component
 (We own the source code)

session 15

=====

Java Configuration : Working with properties file

Lombok API

=====

Java Configuration : Working with properties file

=> properties file holds data in key=val

=> This is used to provide application setup data at runtime.

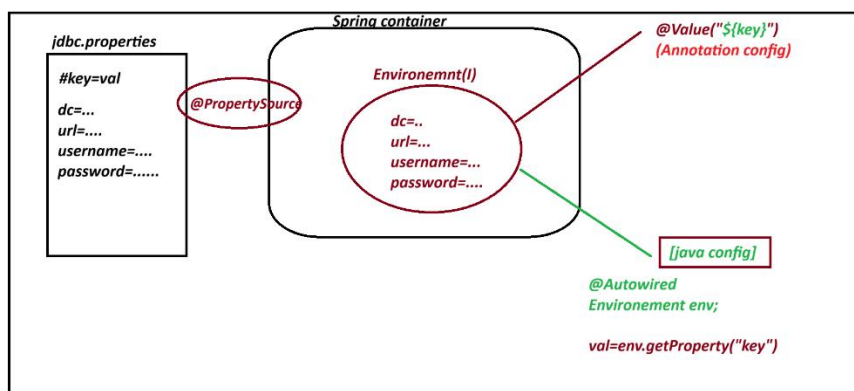
ex : Jdbc Connection details, ORM details, Email Config, Security View Resolvers ..etc

=> .properties file can be loaded using annotation `@PropertySource`

ex : `@PropertySource("classpath:jdbc.properties")`

=> Spring Container all these details into `Environment()` memory spring f/w has given impl class : `StandardEnvironment()`

=> This `Environment()` can be read from container to app using `@Autowired`.



Exmample code

Dbcon-java-config-propertiesS15

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.app</groupId>
```

```
<artifactId>Dbcon-java-config-propertiesS15</artifactId>
```

```
<version>1.0</version>
```

```
<properties>
```

```
<maven.compiler.source>8</maven.compiler.source>
```

```
<maven.compiler.target>8</maven.compiler.target>
```

```
</properties>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework</groupId>
```

```
<artifactId>spring-context</artifactId>
```

```
<version>7.0.5</version>
```

```
<scope>compile</scope>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>jakarta.annotation</groupId>
```

```
<artifactId>jakarta.annotation-api</artifactId>
```

```
<version>2.0.0</version>
```

```
</dependency>
```

```
</dependencies>
```

```
</project>
```

```
-----  
MyDbConnection.java
```

```
-----  
package com.app.bean;
```

```
public class MyDbConnection {
```

```
    private String driver;
```

```
    private String url;
```

```
    private String username;
```

```
    private String password;
```

```
    public MyDbConnection() {
```

```

super();

// TODO Auto-generated constructor stub
}

public String getDriver() {
return driver;
}

public void setDriver(String driver) {
this.driver = driver;
}

public String getUrl() {
return url;
}

public void setUrl(String url) {
this.url = url;
}

public String getUsername() {
return username;
}

public void setUsername(String username) {
this.username = username;
}

public String getPassword() {
return password;
}

public void setPassword(String password) {
this.password = password;
}

@Override
public String toString() {
return "MyDbConnection [driver=" + driver + ", url=" + url + ", username=" + username + ", password=" +
password+ " ]";
}

```

```
}
```

```
-----  
AppConfig.java  
-----
```

```
package com.app.config;
```

```
import org.jspecify.annotations.Nullable;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.context.annotation.PropertySource;
```

```
import org.springframework.core.env.Environment;
```

```
import com.app.bean.MyDbConnection;
```

```
@Configuration
```

```
@PropertySource("classpath:jdbc.properties")
```

```
public class AppConfig {
```

```
@Autowired
```

```
Environment env;
```

```
@Bean
```

```
public MyDbConnection dbObj() {
```

```
MyDbConnection d = new MyDbConnection();
```

```
d.setDriver(env.getProperty("my.app.driver-class"));
```

```
d.setUrl(env.getProperty("my.app.url"));
```

```
d.setUsername(env.getProperty("my.app.username"));
```

```
d.setPassword(env.getProperty("my.app.password"));
```

```
return d;
```

```
}
```

```
-----
```


Test.java

```
package com.app.test;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
import com.app.bean.MyDbConnection;
```

```
import com.app.config.AppConfig;
```

```
public class Test {
```

```
    @SuppressWarnings("resource")
```

```
    public static void main(String[] args) {
```

```
        AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        MyDbConnection ob = ac.getBean("dbObj",MyDbConnection.class);
```

```
        System.out.println(ob);
```

```
    }
```

```
}
```

```
-----  
jdbc.properties
```

```
-----
```

```
# key name can have A-Z a-z dash(-) dot(.) underscore(_)
```

```
my.app.driver-class=Oracle Driver
```

```
my.app.url=JDBC-ORCL:thin
```

```
my.app.username=root
```

```
my.app.password=1234
```

```
}
```

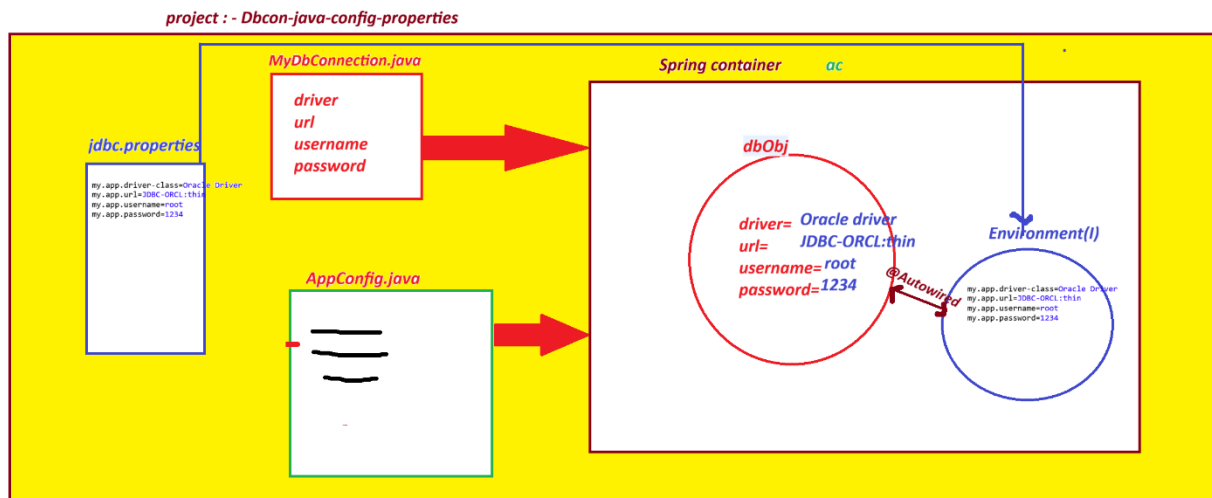
```
-----
```

```
output
```

```
-----
```

MyDbConnection [driver=Oracle Driver, url=JDBC-ORCL:thin, username=root, password=1234]

Diagram



=====

PROJECT Lombok API

=====

=> It is open source Java API, used to generate code if we apply annotations

=> Lombok can generate code like

- Constructor (default, Parameterized)
- setter/getter methods
- toString()
- hashCode()
- equals()

=====

Annotation

Work

@Getter	Generate getter()
@Setter	Generate setter()
@ToString	toString()
@NoArgsConstructor	Default constructor
@AllArgsConstructor	Parameterized constructor
@Data	Getter+Setter+toString+equals+hashCode

@Builder

Builder Patter implement

@RequiredArgsConstructor

Required fields constructor

example:

```
import lombok.Getter;
```

```
import lombok.NoArgsConstructor;
```

```
import lombok.Setter;
```

```
import lombok.ToString;
```

```
@NoArgsConstructor
```

```
@Setter
```

```
@Getter
```

```
@ToString
```

```
public class MyDbConnection {
```

```
    private String driver;
```

```
    private String url;
```

```
    private String username;
```

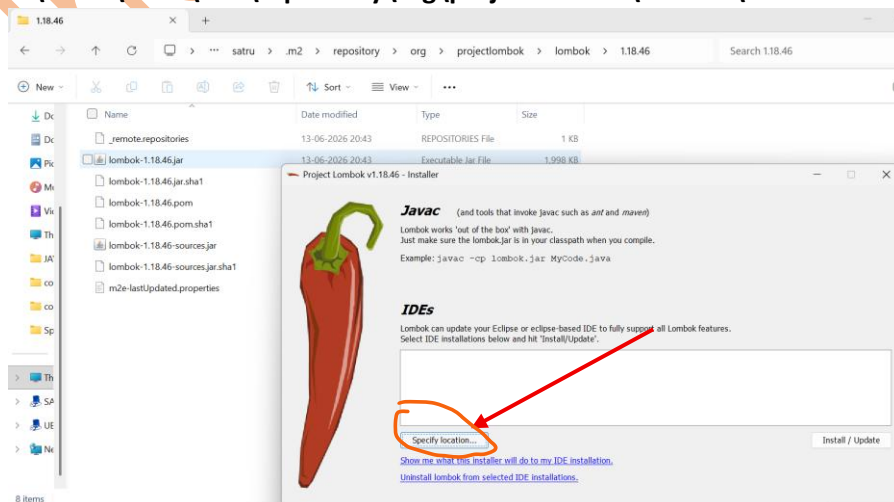
```
    private String password;
```

```
}
```

=====

Go for Lombok Installation

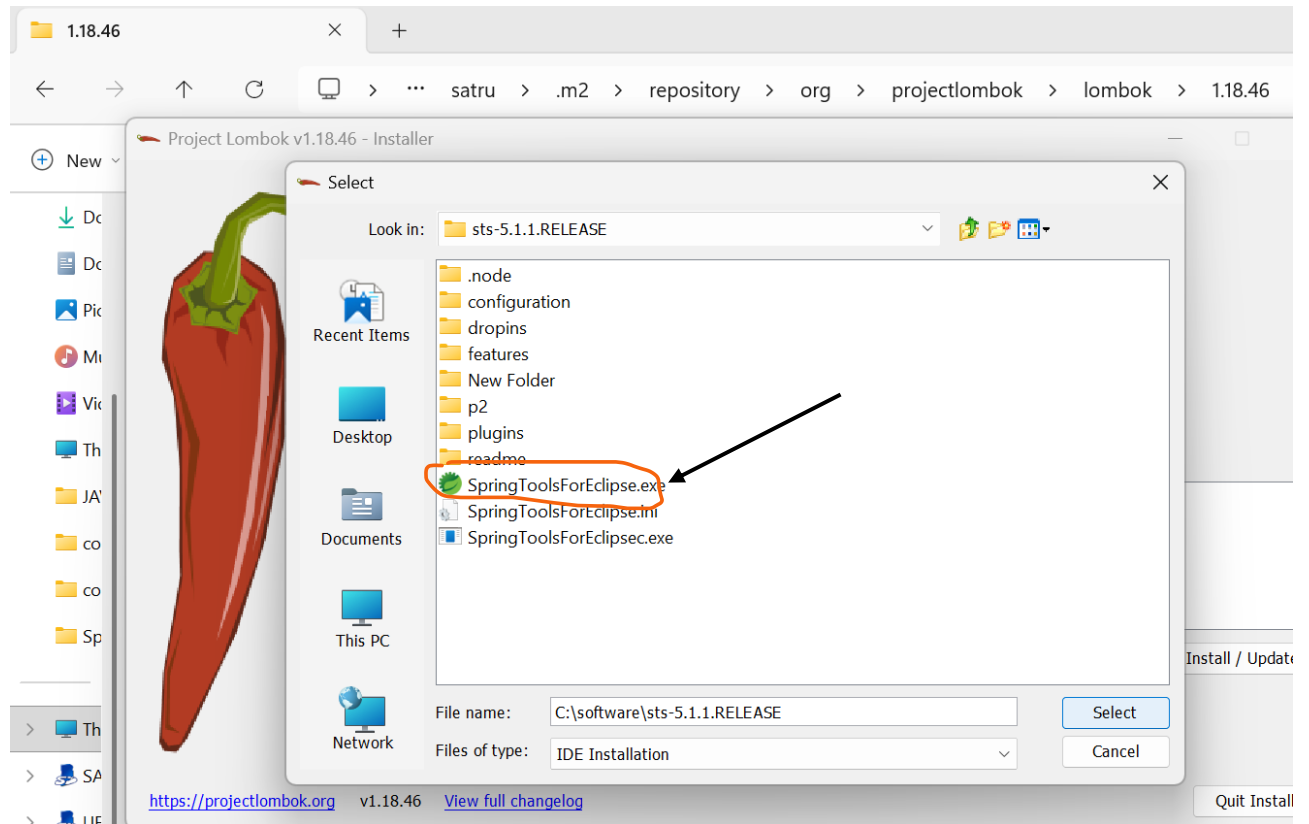
1. C:\Users\satru\.m2\repository\org\projectlombok\lombok\1.18.46



2. double click : lombok-1.18.46

3. wait sometime to detect STS IDE location

If not click on specify location and select sts IDE location



4. click on Install/Update > quite.

5. Open Again

=====

session 16

=====

Spring bean scope:

Scope: - Life time of object present in memory/ How Long object exist in memory.

Spring bean Scope: When bean(object) is created, how long it is present in container.

1. singleton

2. prototype

3. request

4. session

1. singleton(default) : for every bean this is default scope. One object is created per configuration.

It returns single object even if we access it multiple times.

2. prototype : In prototype scope, a new bean instance is created every time the bean is requested from the container.

3. request : web app container new object when a new request;

4. session : web app container creates object when new session is created (Login-Logout) and object is removed when session is destroyed.

1. spring xml Configuration:

Ex program

project name: bean-scope-s16

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.app</groupId>

  <artifactId>bean-scope-s16</artifactId>

  <version>1.0</version>

  <properties>

<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>

  <dependencies>

  <dependency>

<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
```

```
<scope>compile</scope>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.projectlombok</groupId>
```

```
<artifactId>lombok</artifactId>
```

```
<version>1.18.46</version>
```

```
<scope>provided</scope>
```

```
</dependency>
```

```
</dependencies>
```

```
</project>
```

```
-----
```

```
Product.java
```

```
package com.app;
```

```
import lombok.Getter;
```

```
import lombok.Setter;
```

```
import lombok.ToString;
```

```
@Setter
```

```
@Getter
```

```
@ToString
```

```
public class Product {
```

```
private int pid;
```

```
private String pcode;
```

```
}
```

```
-----
```

Test.java

```
package com.app;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {
```

```
    @SuppressWarnings("resource")
```

```
    public static void main(String[] args) {
```

```
        ClassPathXmlApplicationContext ac = new ClassPathXmlApplicationContext("config.xml");
```

```
        /*singleton
```

```
        Product ob1 = ac.getBean("pobj",Product.class);
```

```
        Product ob2 = ac.getBean("pobj",Product.class);
```

```
        Product ob3 = ac.getBean("pobj",Product.class);
```

```
        System.out.println(ob2==ob3);
```

```
        */
```

```
        Product p1 = ac.getBean("pobj",Product.class);
```

```
        Product p2 = ac.getBean("pobj",Product.class);
```

```
        Product p3 = ac.getBean("pobj",Product.class);
```

```
        System.out.println(p1==p2);
```

```
    }
```

```
}
```

```
-----
```

config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="
```

```
        http://www.springframework.org/schema/beans
```

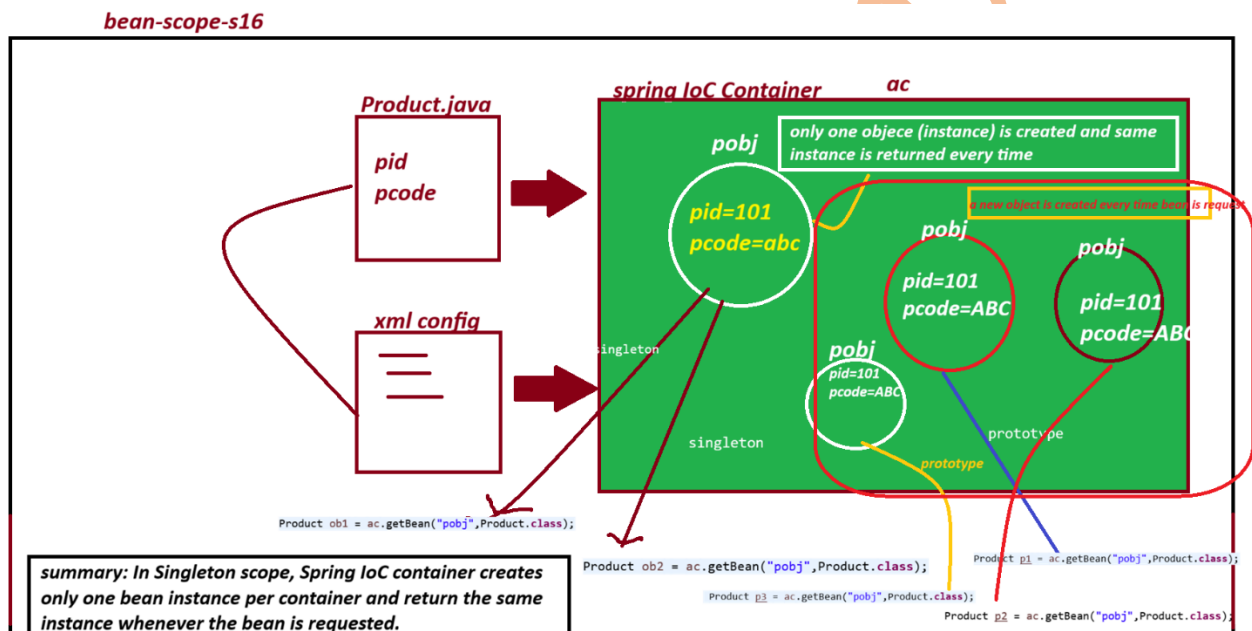
```
        http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```

<bean class="com.app.Product" id="pobj" scope="prototype">
  <property name="pid" value
="101"/>
  <property name="pcode" value="ABC"/>
</bean>
</beans>
=====

```

xml diagram



2. Spring Annotation Configuration

example

project name : bean-scope-annotation-s16

pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
  4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

```



```

<groupId>com.app</groupId>
<artifactId>bean-scope-annotation-s16</artifactId>
<version>1.0</version>
<properties>
<maven.compiler.source>21</maven.compiler.source>
<maven.compiler.target>21</maven.compiler.target>
</properties>

```

```

<dependencies>

```

```

<dependency>

```

```

<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>7.0.5</version>
<scope>compile</scope>
</dependency>

```

```

<dependency>

```

```

<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
<version>1.18.46</version>
<scope>provided</scope>
</dependency>
</dependencies>

```

```

</project>

```

```

-----

```

```

Product.java

```

```

-----

```

```

package com.app;

```

```

import org.springframework.beans.factory.annotation.Value;

```

```

import org.springframework.context.annotation.Scope;

```

```
import org.springframework.stereotype.Component;
```

```
import lombok.Getter;
```

```
import lombok.Setter;
```

```
import lombok.ToString;
```

```
@Getter
```

```
@Setter
```

```
@ToString
```

```
@Component("pobj")
```

```
//@Scope("singleton")
```

```
@Scope("prototype")
```

```
public class Product {
```

```
@Value("101")
```

```
private int pid;
```

```
@Value("ABC")
```

```
private String pcode;
```

```
}
```

```
-----
```

```
Test.java
```

```
package com.app;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class Test {
```

```
@SuppressWarnings("resource")
```

```
public static void main(String[] args) {
```

```
AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
Product p1 = ac.getBean("pobj",Product.class);
```

```
Product p2 = ac.getBean("pobj",Product.class);
```

```
System.out.println(p1==p2);
}
```

```
}
```

```
-----
config.java
```

```
package com.app;
```

```
import org.springframework.context.annotation.ComponentScan;
```

```
@ComponentScan("com.app")
```

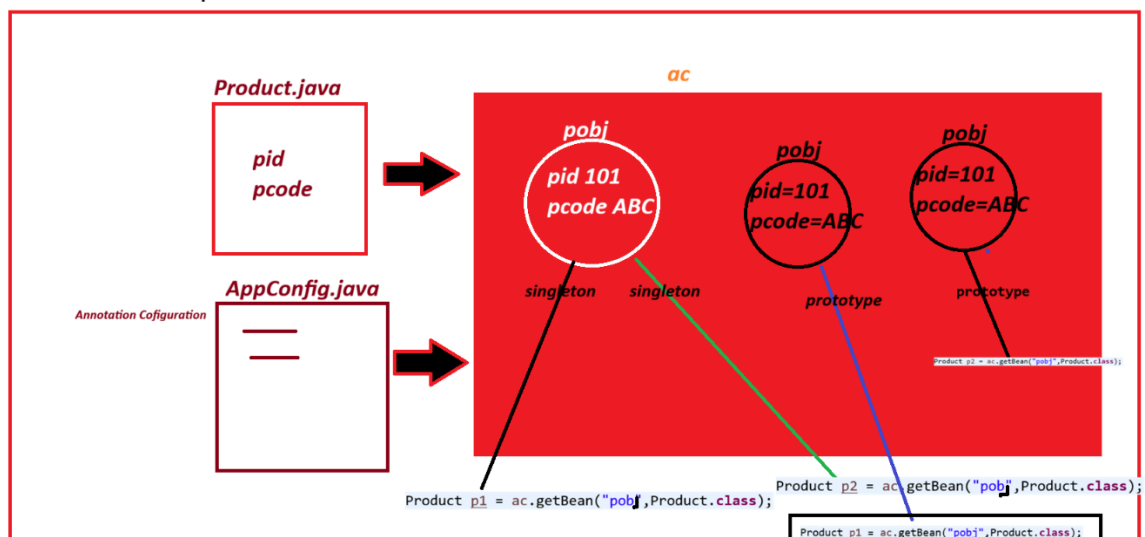
```
public class AppConfig {
```

```
}
```

```
digram
```

```
-----
```

bean-scope-annotation-s16



```
-----
```

```
=====
```

3. Spring java configuration

example bean-scope-java-config-s16

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.app</groupId>

  <artifactId>bean-scope-java-config-s16</artifactId>

  <version>1.0</version>

  <properties>

<maven.compiler.source>21</maven.compiler.source>

<maven.compiler.target>21</maven.compiler.target>

</properties>

  <dependencies>

    <dependency>

      <groupId>org.springframework</groupId>

      <artifactId>spring-context</artifactId>

      <version>7.0.5</version>

      <scope>compile</scope>

    </dependency>

    <dependency>

      <groupId>org.projectlombok</groupId>

      <artifactId>lombok</artifactId>

      <version>1.18.46</version>

      <scope>provided</scope>

    </dependency>

  </dependencies>

</project>
```

Product.java

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
```

```
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.app</groupId>
```

```
  <artifactId>bean-scope-java-config-s16</artifactId>
```

```
  <version>1.0</version>
```

```
  <properties>
```

```
    <maven.compiler.source>21</maven.compiler.source>
```

```
    <maven.compiler.target>21</maven.compiler.target>
```

```
  </properties>
```

```
  <dependencies>
```

```
    <dependency>
```

```
      <groupId>org.springframework</groupId>
```

```
      <artifactId>spring-context</artifactId>
```

```
      <version>7.0.5</version>
```

```
      <scope>compile</scope>
```

```
    </dependency>
```

```
    <dependency>
```

```
      <groupId>org.projectlombok</groupId>
```

```
      <artifactId>lombok</artifactId>
```

```
      <version>1.18.46</version>
```

```
      <scope>provided</scope>
```

```
    </dependency>
```

```
  </dependencies>
```

```
</project>
```

AppConfig.java

```
package com.app;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.context.annotation.Scope;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean
```

```
    //@Scope("singleton")
```

```
    @Scope("prototype")
```

```
    public Product pObjj(){
```

```
        Product p = new Product();
```

```
        p.setPid(101);
```

```
        p.setPcode("ABC");
```

```
        return p;
```

```
    }
```

```
}
```

```
-----
```

```
Test.java
```

```
package com.app;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
public class Test {
```

```
    @SuppressWarnings("resource")
```

```
    public static void main(String[] args) {
```

```
        AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        Product p1 = ac.getBean("pobj",Product.class);
```

```
        Product p2 = ac.getBean("pobj",Product.class);
```

```
        tln(p1==p2);
```

```
System.out.prin
```

}

}

output

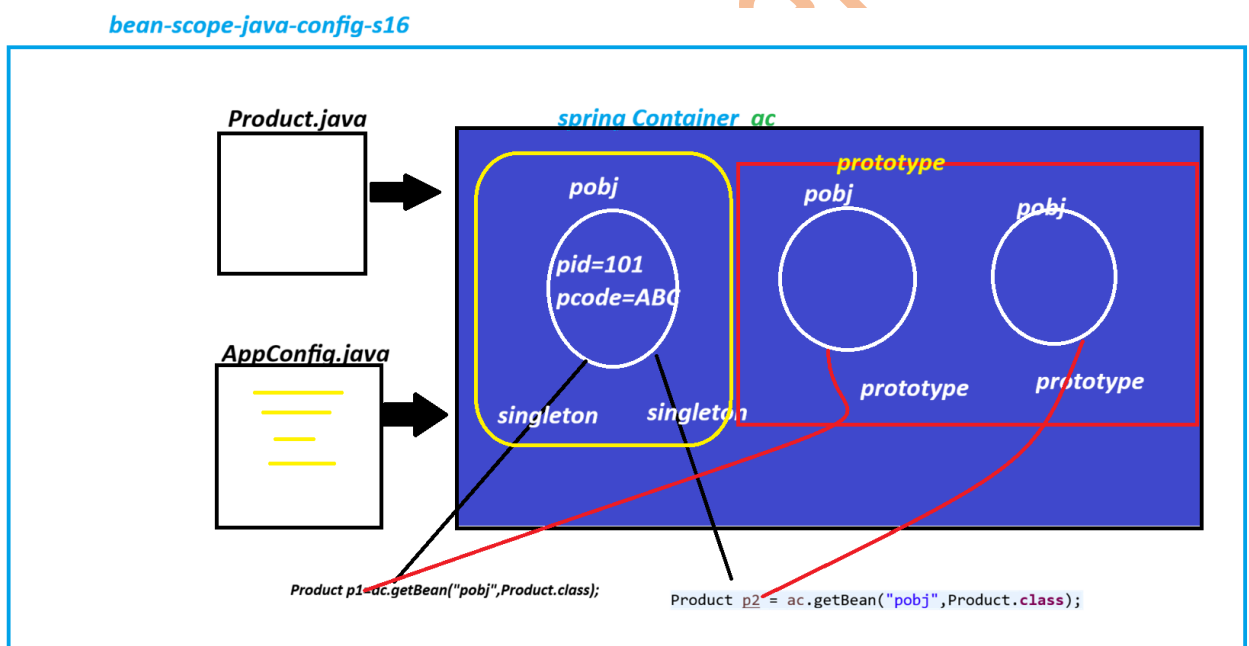
for prototype

FROM CONSTRUCTOR

FROM CONSTRUCTOR

false

diagram



Session 17

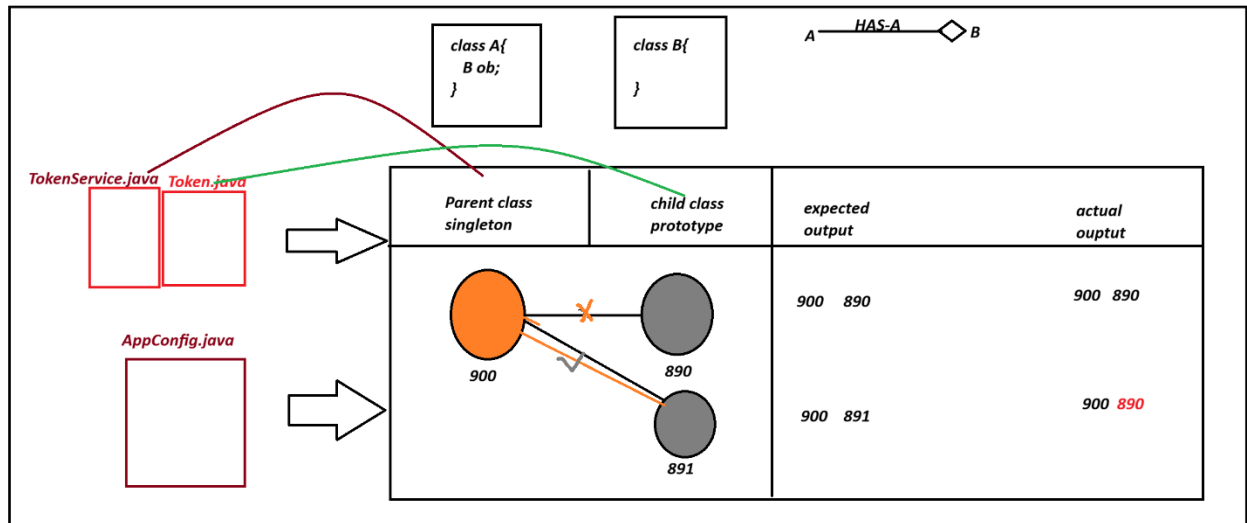
Spring Bean Scope - Lookup Method Injection (LMI)

problem statement:

If two Spring beans(classes) are connected using Reference Type(HAS-A) and child class(independent) scope is prototype and parent class(Dependent) scope is singleton. Then they may not get linked as expected .

or Always parent links to first child object.

diagram



code

project name bean-scope-LMI-s17

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.app</groupId>
  <artifactId>bean-scope-LMI-s17</artifactId>
  <version>1.0</version>
  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>
```


<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>7.0.5</version>

<scope>compile</scope>

</dependency>

<dependency>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<version>1.18.46</version>

<scope>provided</scope>

</dependency>

</dependencies>

</project>

Spring bean

Token.java

package com.app.bean;

import java.util.Random;

import org.springframework.context.annotation.Scope;

import org.springframework.stereotype.Component;

@Component

@Scope("prototype")

public class Token {

private int token;

```
public Token() {token = new Random().nextInt(999);  
}
```

@Override

```
public String toString() {  
    return "Token [token=" + token + "];"  
}
```

```
}
```

Spring bean

TokenService.java

```
package com.app.bean;
```

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.annotation.Scope;  
import org.springframework.stereotype.Component;
```

@Component

@Scope("singleton")

```
public class TokenService {
```

@Autowired

```
private Token tokenOb;
```

```
public Token getTekenOb() {  
    return tokenOb;  
}
```

@Override

```

public String toString() {
    return "TokenService [tokenOb=" + tokenOb + "];"
}

```

```

}

```

```

=====

```

AppConfig.java

```

package com.app.config;

```

```

import org.springframework.context.annotation.ComponentScan;

```

```

@ComponentScan("com.app")

```

```

public class AppConfig {

```

```

}

```

```

-----

```

Test.java

```

package com.app.test;

```

```

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

```

```

import com.app.bean.TokenService;

```

```

import com.app.config.AppConfig;

```

```

public class Test {

```

```

@SuppressWarnings("resource")

```

```

public static void main(String[] args) {

```

```

    AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);

```

```

    TokenService ts1 = ac.getBean("tokenService",TokenService.class);

```

```

    System.out.println(ts1);

```

```

    System.out.println("Token Serive HS : "+ ts1.hashCode());
}

```

```
System.out.println("Token HS :"+ ts1.getTekenOb().hashCode());
```

```
TokenService ts2 = ac.getBean("tokenService",TokenService.class);
```

```
System.out.println(ts2);
```

```
System.out.println("Token Serive HS : "+ ts2.hashCode());
```

```
System.out.println("Token HS :"+ ts2.getTekenOb().hashCode());
```

```
}
```

```
}
```

```
=====
```

ouput

```
-----
```

```
TokenService [tokenOb=Token [token=507]]
```

```
Token Serive HS : 178049969
```

```
Token HS :333683827
```

```
TokenService [tokenOb=Token [token=507]]
```

```
Token Serive HS : 178049969
```

```
Token HS :333683827
```

```
=====
```

Solution:

This solution is given by spring f/w. porgrammer need not to implement any logic. But need to provide some details to container.

-> code Modifications come at parent class only.

1. define one public method with empty body.
2. Return type of such method should be child type
3. and return null statement
4. Add one annotation over method @Lookup
5. class this method some where is code (ex: getter method)

--- solution code-----

```
-----
```

TokenService.java

```
package com.app.bean;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Lookup;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;
```

```
@Component
```

```
@Scope("singleton")
```

```
public class TokenService {
```

```
@Autowired
```

```
private Token tokenOb;
```

```
// problem solution
```

```
@Lookup
```

```
public Token linkNewChildObj() {
```

```
return null;
```

```
}
```

```
public Token getTekenOb() {
```

```
this.tokenOb = linkNewChildObj();
```

```
return tokenOb;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
return "TokenService [tokenOb=" + tokenOb + "];"
```

```
}
```

```
}
```

```
=====
```

```
solution output
```

```
-----
```

TokenService [tokenOb=Token [token=997]]

Token Service HS : 130668770

Token HS :330739404

TokenService [tokenOb=Token [token=349]]

Token Service HS : 130668770

Token HS :361398902

=====

Circular Dependency

=====

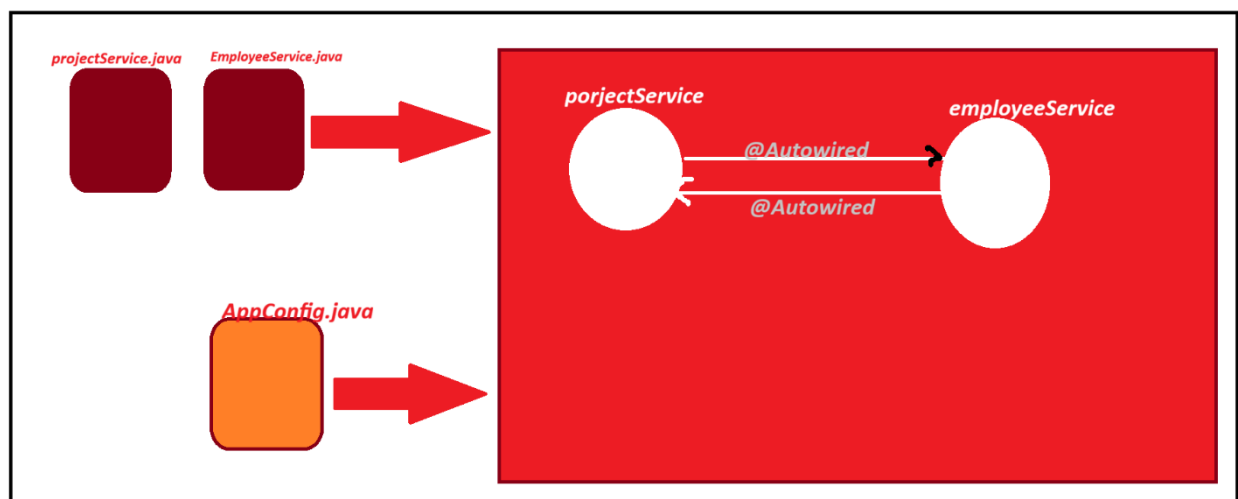
=> If two classes are connected using Reference (HAS-A) Relation then objects are created using default constructors first and linked using set methods one after another.

such process is called as circular Dependency and handled by Spring container.

:- do not override toString() method in both classes.

if we do, then toString() method are called in loop from both classes then results :

java.lang.StackOverflowError



@Component

```
public class A{
```

```
    @Autowired
```

```

    private B b;
}
@Component
public class B{
    @Autowired
    private A a;
}

```

project name :
circular-dependency-s17

pom.xml

some of above

EmployeeService.java

package com.app.bean;

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

```

```

@Component
public class EmployeeService {

    @Autowired
    private ProjectService pservice;

}

```

ProjectService.java

package com.app.bean;

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

```

@Component

```
public class ProjectService {
```

@Autowired

```
private EmployeeService eService;
```

@Override

```
public String toString() {
```

```
return "ProjectService [eService=" + eService + "];
```

```
}
```

```
}
```

AppConfig.java

```
package com.app.config;
```

```
import org.springframework.context.annotation.ComponentScan;
```

@ComponentScan("com.app")

```
public class AppConfig {
```

```
}
```

Test.java

```
package com.app.test;
```

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
```

```
import com.app.bean.ProjectService;
```

```
import com.app.config.AppConfig;
```



```
public class Test {
```

```
    @SuppressWarnings("resource")
```

```
    public static void main(String[] args) {
```

```
        AnnotationConfigApplicationContext ac = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        ProjectService ps = ac.getBean("projectService",ProjectService.class);
```

```
        System.out.println(ps);
```

```
    }
```

```
}
```

```
=====
```

output

ProjectService [eService=com.app.bean.EmployeeService@2e4b8173]

```
=====
```

session 18 (2. SPRING BOOT)

```
=====
```

Spring Framework

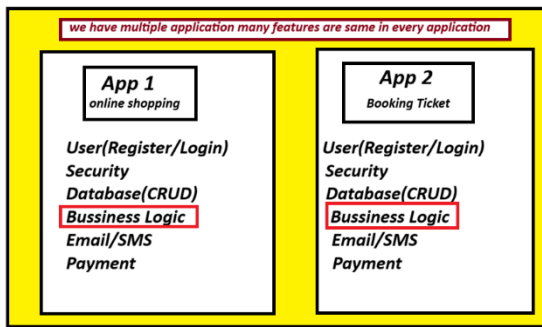
Spring Boot

Spring Framework:

=> It is mainly used to develop web/webservices application.

=> Here, programmer has to define configuration [input to spring container.] which is very lengthy and started from line 1.

diagram



Disadvantage

- > many application have some features.
- > we have to write same code again and again.
- > It increase development time.
- > It is boring and error-prone.
- > we need an easier and faster solution.

Spring Framework (OLD WAY)

```
<bean id="dsObj" class="DriverManagerDataSource">
  <property name="driverClassName">
    <value>OracleDriver</value>
  </property>
  <property name="url">
    <value>jdbc:orcl:thin:@localhost:1521:orcl</value>
  </property>
  <property name="username">
    <value>indiafreeinternship</value>
  </property>
  <property name="password">
    <value>root</value>
  </property>
</bean>
```

diagram

Spring Framework (OLD WAY)
In spring Framework, we have to do a lot of manual configuration.
Example: Database Connection (XML)

```
<bean id="dsObj" class="DriverManagerDataSource">
  <property name="driverClassName">
    <value>OracleDriver</value>
  </property>
  <property name="url">
    <value>jdbc:orcl:thin:@localhost:1521:orcl</value>
  </property>
  <property name="username">
    <value>indiafreeinternship</value>
  </property>
  <property name="password">
    <value>root</value>
  </property>
</bean>
```

->In Spring framework, we need to write a lot of XML configuration.

-> we have to configure every component manually.

-> for example : database , Security, Transaction, etc..

-> So much configuration just for one connection!

-> Development becomes slow and comples.

Spring Boot :-

-Spring boot is a spring based framework that provides Auto Configuration, Starter Dependencies and Embedded server suport to

reduce developement time and boilerplate code.

-Spring boot provides all common things in a parent project.

-It includes Auto configuration, Common Classes, Starter JARs, Embedded Server, Database Setup, etc...

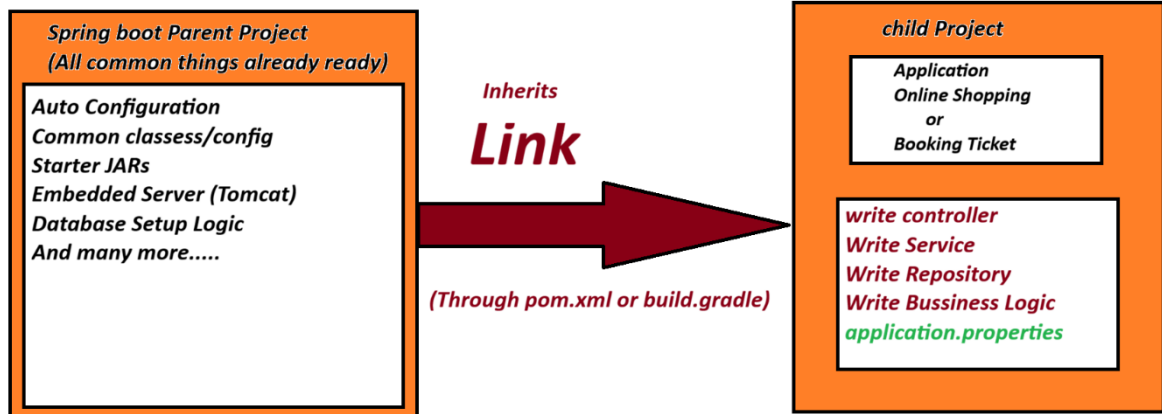
-We create our own project (Child project) and link it with parent.

-parent gives all these facilites automatically.

-We just write our bussniess logic and application.properties.

-This saves a lot of time and efforts.

Spring Boot (by Pivotal Team) provide everything ready in a Parent project.
We just link our project and start coding!



=====

Starter Dependencies (STARTERS)

=====

Starter are ready-made JARs.

They include common code, libraries and configuration.

we just add the required started in pom.xml or build.gradle.

It includes everything we need for that features.

Diagram

starter Dependencies (Starters)	
Starters are pre-build JARs They contain common code and configuration	
Starter Name	Use
spring-boot-starter-web	Web(REST APIs, MVC)
spring-boot-starter-jdbc	Database Connectivity (JDBC)
spring-boot-starter-data-jpa	JPA+Hibernate
spring-boot-starter-security	Security(Login, Auth)
spring-boot-starter-mail	Email Sending
spring-boot-starter-json	JSON Support
..and many more	

Difference between Spring Framework and Spring Boot

Diagram

SPRING FRAMEWORK vs SPRING BOOT	
<u>Spring framework</u>	<u>Spring Boot</u>
Manual configuration	Auto configuration
XML based Setup	Convention over configuration
More Coding	Less Coding
More Time	Less Time(Fast Development)
No Starter Concept	Starter Dependencies
External Server(Tomcat, etc)	Embedded server(Tomcat,jetty)
complex for beginner	Easy for Beginners

=====

session 19

=====

=> Spring boot is a spring based project that provides AutoConfigurations
(Reduce common code written in Spring f/w)

=> Spring boot has provided starters (pre-defined JARs for every concept)

ex: spring-boot-starter-jdbc

spring-boot-starter-data-jpa

.... etc

=> Spring Boot applications must connect with parent Project our project is known as child project.

=> If we use maven project there will be pom.xml

for Gradle build.gradle files exist.

=> Spring boot do not support XML configuration.

we need to work on java and Annotation Configuration.

xml is slow compared to java/annoation.

=> Spring boot has provided Embeeded Database (H2, HyperSQL, Derby)

These are also called as In-Memory Database(No Download + No Install)

Recomanded to use dev/test Environment. Not in Production.

=> Spirng boot has provided Embedded Webserver:

(Apache Tomcat(default), Eclipse Jetty, JBoss Undertow)

to run our application (No download + No Install)

<https://start.spring.io/>

Spring Initializer

=> This website provides support to create spring boot application.

-> Even if we use Any IDE (Eclipse, STS, VScode, IntelliJDea) that should be finally connected to spring Initializer.

=====

session 20

=====

=====

Spring Boot - Project Structure and basics.

=====

step to create project:

> File> new > Spring Starter project> Enter Details

Type of files : There are mainly 3 types of Files in Every Spring Boot application, gives as:

1. Starter class /Main Class

=> This is an entry class in execution.

It create spring Container (new Container : ApplicationContext(I)

Impl: AnnotationConfigApplicationContext(C)

=> It load all classes using basePackage All the classes which are in basePackage or subPackage are scanned by Spring Container and create objects.

=> provide data, read properties file, inject data, link object..etc

2. Input File (application.properties/application.yml)

=> This file is used to pass input to container objects.

=>It holds data in Key=value format

=>keys are pre-defined here, we can pass our own key(custom keys).

=>Internally every key is a variable

=> we can find all pre-defined key list at:

<https://docs.spring.io/spring-boot/appendix/application-properties/index.html#appendix.application-properties.cache>

=> names of these files are fixed.

But we can even provide our custom properties files.

=>A key name can have symbols like dot(.) dash(-) and underscore(_) No other symbols allowed

ex : spring.datasource.driver-class-name=

my.app-name_model=

3. pom.xml

POM Stand for Porject Object Model.

=> The pom.xml file is the configuration file or a Maven project. It contains project information, dependencies, plugins, properties, and build configuration required to build and manage the application.

Advantages of pom.xml

Easy dependency management

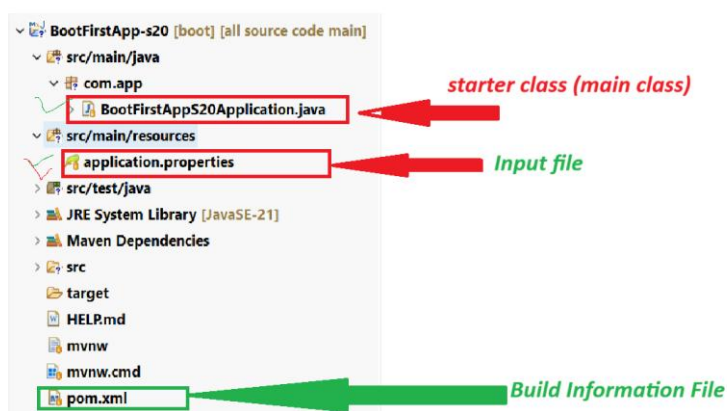
Automatic Library download

Easy project build process.

Better version management

Reduce manual configuration.

Diagram



session 21

=====

=====

Runners:

=>Runners are used to execute any logic only one time, while starting the application (i.e after creating the spring container);

example:

Setup Data in Database.

insert RolesTable(ADMIN, USER, GUEST)..etc

- Load File and read data..etc

-Test some logic..etc

How Runners work in Spring Boot?

=> we need to define one class that should implement interface CommandLineRunner.

=> we need to override abstract method run() and define our logic.

=> Main class/starter class, after creating container, executing all runners classes.

=> we can define multiple runners, and we can provide our own execution order using @Order(value) annotation.

=> @Order(_) lowest value is executed first. if we did not specify any Order then executed at last.

=====

ex Code

project Name : BootRunnerEx-s21

MessageRunner.java

package com.app.runner;

import org.springframework.boot.CommandLineRunner;

import org.springframework.core.annotation.Order;

import org.springframework.stereotype.Component;

@Component

@Order(5)

public class MessageRunner implements CommandLineRunner{

@Override

```
public void run(String... args) throws Exception {  
    System.out.println("FROM MESSAGE RUNNER");
```

```
}
```

```
}
```

```
-----
```

EmailRunner.java

```
package com.app.runner;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.core.annotation.Order;
```

```
import org.springframework.stereotype.Component;
```

@Component

@Order(2)

```
public class EmailRunner implements CommandLineRunner {
```

@Override

```
public void run(String... args) throws Exception {  
    System.out.println("FROM EMAIL RUNNER");
```

```
}
```

```
}
```

```
-----
```

JdbcRunner.java

```
package com.app.runner;
```

```
import org.springframework.boot.CommandLineRunner;
```

```

import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Component
@Order(3)
public class JdbcRunner implements CommandLineRunner {

    @Override
    public void run(String... args) throws Exception {
        System.out.println("FROM JDBC RUNNER");
    }

}

```

output

FROM EMAIL RUNNER

FROM JDBC RUNNER

FROM MESSAGE RUNNER

=====

SESSION 22

=====

=====

@Value : This annotation is used to read data from Properties file into variable.

If we have n-variable in class then we need to provide n-@Value annotation.

@ConfigurationProperties:

=> It is used to load multiple key=value data into variable

=> Must provide one common prefix for key name.

=> Same prefix must be matched while loading keys in class

=> key name and variable name must be matching.

=> Must generate setter/getter methods.

=> If we don't follow above rules then spring boot will not load key=val data into variables. No Error, No Exception. variable holds default values.

example project

BootConfigPropsEx-s22

1. Spring Bean

package com.app.service;

import org.springframework.boot.context.properties.ConfigurationProperties;

import org.springframework.stereotype.Component;

import lombok.Data;

@Data

@Component

@ConfigurationProperties(prefix = "my.app")

public class EmailService {

private String host;

private int port;

private String username;

private boolean active;

}

2. application.properties

spring.application.name=BootConfigPropsEx-s22

my.app.host=GmailServer

my.app.port=586

my.app.username=IndiaFreeInternship

my.app.active=false

3. Runner class

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import com.app.service.EmailService;
```

```
@Component
```

```
public class TestObjRunner implements CommandLineRunner {
```

```
    @Autowired
```

```
    private EmailService emailService;
```

```
    @Override
```

```
    public void run(String... args) throws Exception {
```

```
        System.out.println(emailService);
```

```
    }
```

```
}
```

output

```
EmailService(host=GmailServer, port=586, username=IndiaFreeInternship, active=false)
```

@ConfigurationProperties with Collections

=>It Support loading Collections data from properties file of type

List,Set,Array,Map and Properties

=> for List/Set/Array

syntax is

`prefix.variable[index]=value`

=> For map/Properties syntax is

`prefix.variable.mapKey=mapValue`

=> @ConfigurationProperties even support loading complex reference type data using `prefix.refVariable.variable=value` syntax.

Example project BootConfigPropsEx-s22

=====

1. Spring bean

`package com.app.service;`

`import java.util.Map;`

`import java.util.Properties;`

`import org.springframework.boot.context.properties.ConfigurationProperties;`

`import org.springframework.stereotype.Component;`

`import lombok.Data;`

`@Data`

`@Component`

`@ConfigurationProperties(prefix = "my.app")`

`public class EmailService {`

`private String host;`

`private int port;`

`private String username;`

`private boolean active;`

```
// private List<String> models;  
//private Set<String> models;  
private String[] models;  
//private Map<String,String> data;  
private Properties data;
```

```
private Message mob;
```

```
}
```

```
-----
```

2. Spring bean

```
package com.app.service;
```

```
import lombok.Data;
```

```
@Data
```

```
public class Message {
```

```
private String code;
```

```
private String details;
```

```
}
```

```
-----
```

```
application.properties
```

```
spring.application.name=BootConfigPropsEx-s22
```

```
my.app.host=GmailServer
```

```
my.app.port=586
```

```
my.app.username=IndiaFreeInternship
```

```
my.app.active=false
```

my.app.models[0]=M1

my.app.models[1]=M2

my.app.models[2]=M3

my.app.models[3]=M2

my.app.data.C1=ABC

my.app.data.C2=XYZ

my.app.data.C3=pqr

my.app.mob.code=C1325

my.app.mob.details=SampleMobile

TestObjRunner.java

package com.app.runner;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype.Component;

import com.app.service.EmailService;

@Component

public class TestObjRunner implements CommandLineRunner {

 @Autowired

 private EmailService emailService;

 @Override

 public void run(String... args) throws Exception {

 System.out.println(emailService);

 }

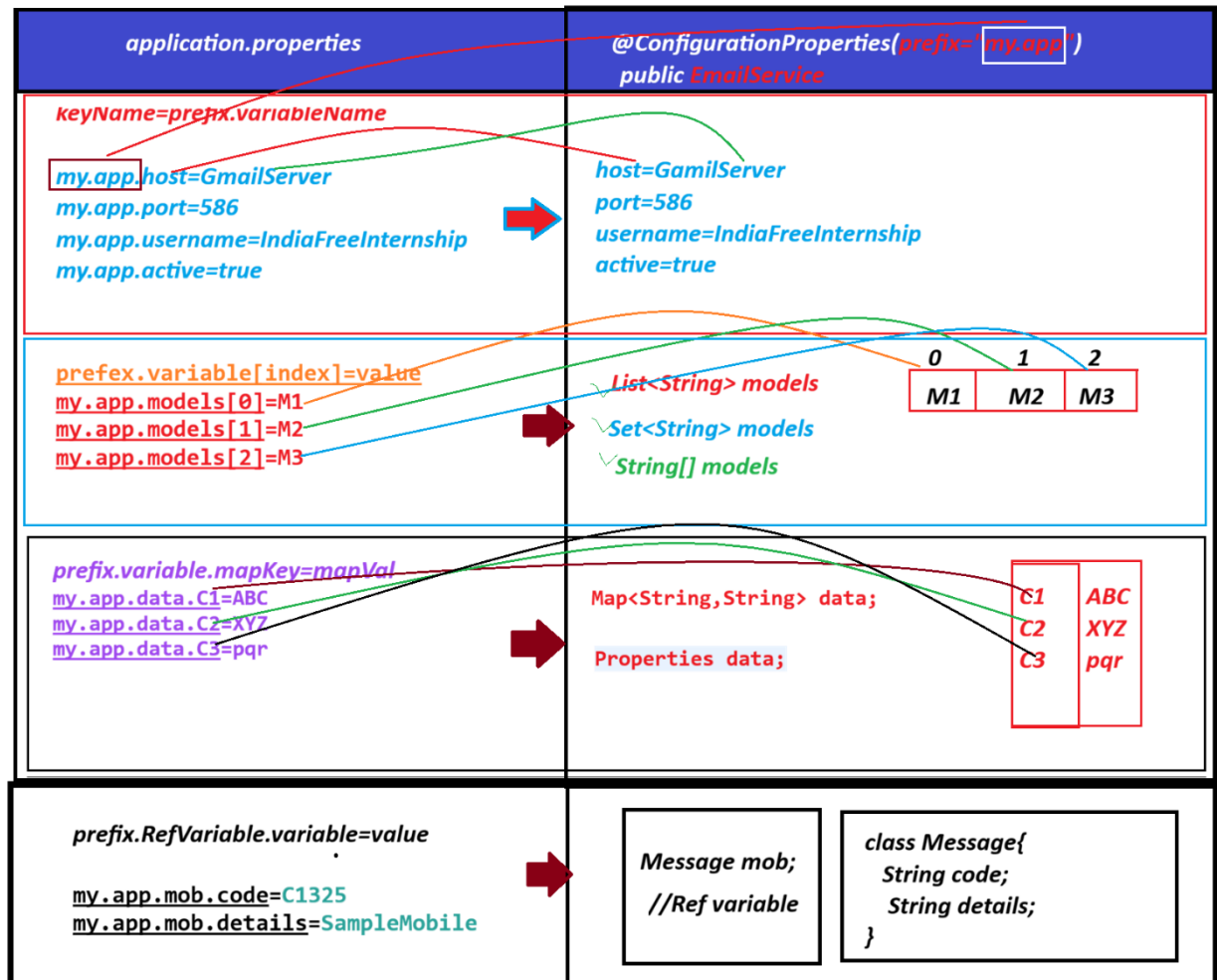
}

output

EmailService(host=GmailServer, port=586, username=IndiaFreeInternship, active=false, models=[M1, M2, M3, M2], data={C3=pqr, C1=ABC, C2=XYZ}, mob=Message(code=C1325, details=SampleMobile))

=====

Diagram



=====

=====

YAML (Yes Another Markup Language)

YAMALiant Language

=>YAML is a new way of represening KEY=VAL paris.

=> In spring boot we can load data using Properties file and YAML file

=> YAML file extension is .yaml

=> In YAML file we do not write duplicate prefix/levels.

=> It is more readable, faster in parsing, supports single file profiles.

=> snake YAML (Jar name)

=====

example project BootConfigYAMLEx-s22

1. Spring bean

```
package com.app.service;
```

```
import java.util.Map;
```

```
import java.util.Properties;
```

```
import org.springframework.boot.context.properties.ConfigurationProperties;
```

```
import org.springframework.stereotype.Component;
```

```
import lombok.Data;
```

```
@Data
```

```
@Component
```

```
@ConfigurationProperties(prefix = "my.app")
```

```
public class EmailService {
```

```
    private String host;
```

```
    private int port;
```

```
    private String username;
```

```
    private boolean active;
```

```
    // private List<String> models;
```

```
    //private Set<String> models;
```

```
    //private String[] models;
```

```
    //private Map<String,String> data;
```

```
//private Properties data;
```

```
//private Message mob;
```

```
}
```

```
-----
```

2. Spring bean

```
package com.app.service;
```

```
import lombok.Data;
```

```
@Data
```

```
public class Message {
```

```
private String code;
```

```
private String details;
```

```
}
```

```
-----
```

```
application.yml
```

```
my:
```

```
  app:
```

```
    host: GmailServer
```

```
    port: 586
```

```
    username: IndiaFreeInternship
```

```
    active: true
```

```
-----
```

```
TestObjRunner.java
```

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
```

```
import com.app.service.EmailService;
```

```
@Component
```

```
public class TestObjRunner implements CommandLineRunner {
```

```
    @Autowired
```

```
    private EmailService emailService;
```

```
    @Override
```

```
    public void run(String... args) throws Exception {
```

```
        System.out.println(emailService);
```

```
    }
```

```
}
```

output

EmailService(host=GmailServer, port=586, username=IndiaFreeInternship, active=true)

=====

=====

session 23

YAML (Yet Another Markup Language)

YAMALiant Language

=>YAML is a new way of represening KEY=VAL paris.

=> In spring boot we can load data using Properties file and YAML file

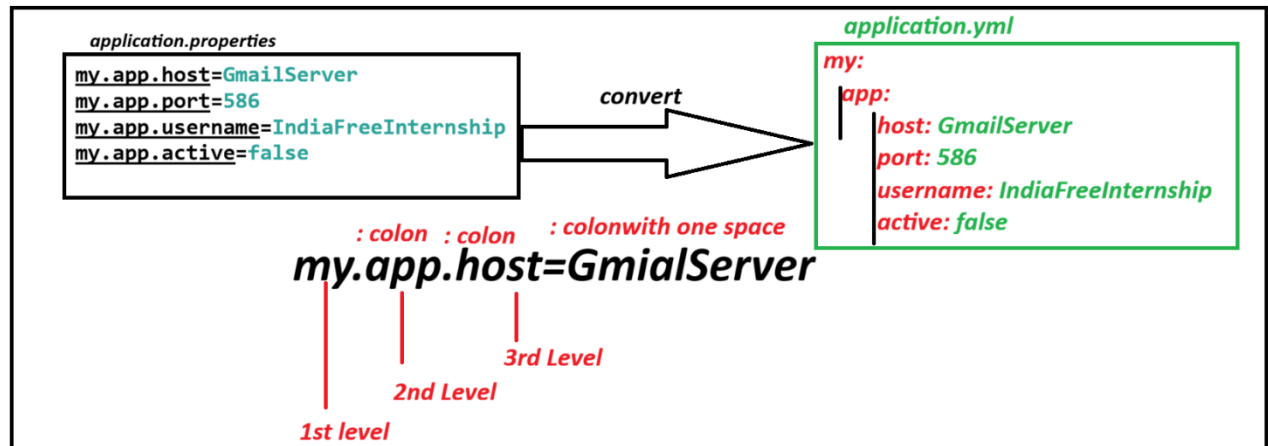
=> YAML file extension is .yaml

=> In YAML file we do not write duplicate prefix/levels.

=> It is more readable, faster in parsing, supports single file profiles.

=> snake YAML (Jar name)

Diagram



Notes:

1. Use symbols colon (:) in the place of dot(.) and equals(=)
2. After every colon (new level) change line for next level.
3. we need to provide spaces for every new level (not for 1st level)
4. Space count must match proper indentation for the given level.
5. Must provide colon and space before value:

=> if we do not follow space rules or any other then spring boot throws (Snake YAML Exception) : ScannerException.

application.properties

my.app.driver=Oracle

my.app.url=JDBC-ORCL

my.app.model=NEW

application.yaml

my:

app:

driver: Oracle

url: JDBC-ORCL

model: NEW

=====

For List/Set/Array Format: (here dash - indicates index number)

key:

- value1

- value2

- value3

=====

For Map/ Properties /ref Type

mapKey: mapValue

=====

Example program:

BootConfigYAMLEx-2-s23

1 Spring bean

ProductInfo.java

package com.app.bean;

import java.util.List;

import java.util.Map;

import org.springframework.boot.context.properties.ConfigurationProperties;

import org.springframework.stereotype.Component;

import lombok.Data;

@Data

@Component

```
@ConfigurationProperties(prefix = "my.app")
```

```
public class ProductInfo {
```

```
private String pcode;
```

```
private int pid;
```

```
private double pcost;
```

```
private List<String> colors;
```

```
private Map<String, String> models;
```

```
private Vendor vob;
```

```
}
```

```
-----
```

2. Spring Bean

```
package com.app.bean;
```

```
import lombok.Data;
```

```
@Data
```

```
public class Vendor {
```

```
private int vid;
```

```
private String vcode;
```

```
}
```

```
-----
```

3. Runner class

```
TestProductRunner.java
```

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import com.app.bean.ProductInfo;
```

```
@Component
```

```
public class TestProductRunner implements CommandLineRunner {
```

```
    @Autowired
```

```
    private ProductInfo productInfo;
```

```
@Override
```

```
public void run(String... args) throws Exception {
```

```
    System.out.println(productInfo);
```

```
}
```

```
}
```

application.yml

```
my:
```

```
  app:
```

```
    pcode: hp360
```

```
    pid: 101
```

```
    pcost: 60000
```

```
    colors:
```

```
      - RED
```

```
      - GREEN
```

```
      - PINK
```

```
    models:
```

```
      M1: ABC
```

```
      M2: MNO
```

```
      M3: XYZ
```

```
  vob:
```

```
    vid: 8888
```

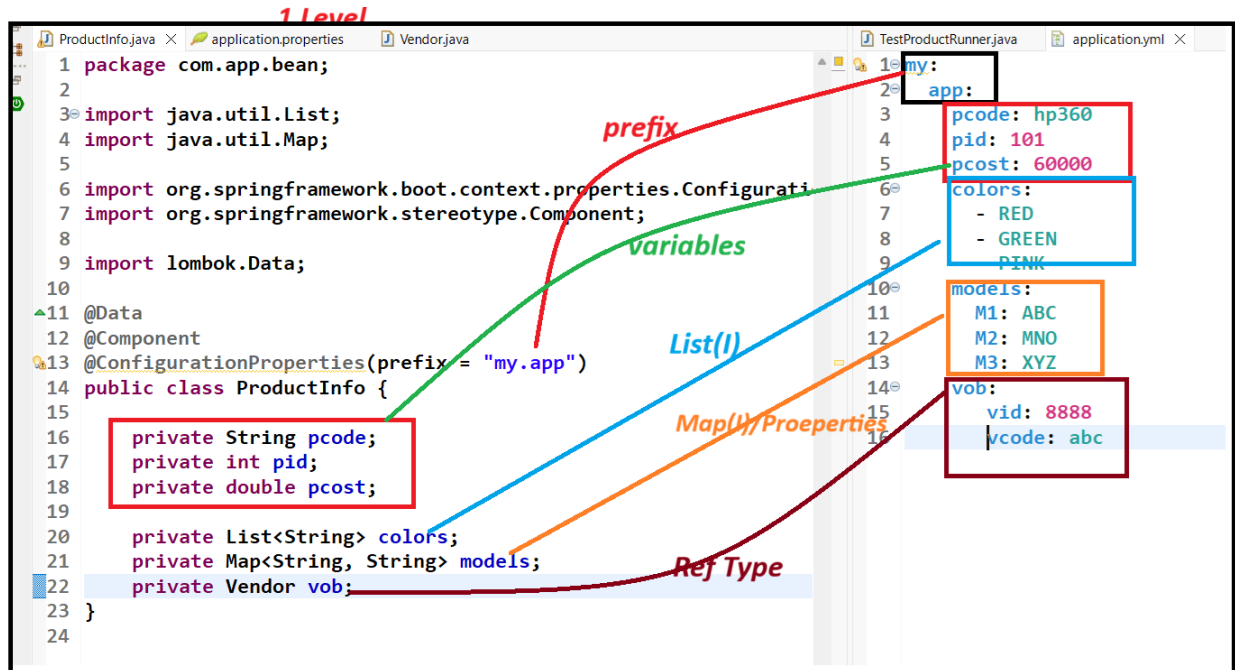
```
    vcode: abc
```

```
=====
```

output

ProductInfo(pcode=hp360, pid=101, pcost=60000.0, colors=[RED, GREEN, PINK], models={M1=ABC, M2=MNO, M3=XYZ}, vob=Vendor(vid=8888, vcode=abc))

Diagram



Example 1

application.properties

my.app.test=A

spring.boot.version=4.1.0

my.app.model=ONE

spring.boot.format=MAVEN

application.yml

my:

app:

test: A

model: ONE

spring:

boot:

version: 4.1.0

format: MAVEN

Example 2

application.properties

my.app.version=4.0

my.test.model=New

my.app.format=OK

my.test.info=WELCOME

application.yml

my:

app:

version: 4.0

format: OK

test:

model: New

infor: WELCOME

=====

Online converter

<https://www.xkit.io/devtools/properties-to-yaml>

=====

session 24

=====

=====

YAML : complex Tyes(List/Map/Ref Type)

List/Set/Array :

prefix:

variable:

- value1 # dash<space>value

- value2

- value3

Map/Properties

prefix:

variable:

mapValue # key:<space>value

mapKey:

mapValue

mapKey:

prefix:

hasAVariable:

variable: value

variable: value

=====

Q) can we define both properties and yaml file in springboot application?

Ans> YES, We can define both application.properties and applicaton.yml

Q) what is both file having same key name and different values?

Ans> Always spring Container takes value from application.properties with high prirority.

example program

name : BootConfigYAMLEx-s24

dependency : Lombok

1. Spring bean

package com.app.bean;

import lombok.Data;

@Data

public class Publisher {

```

pid;

code;

active;
}
-----

package com.app.bean;

import java.util.Map;
import java.util.Set;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

import lombok.Data;

@Data
@Component
@ConfigurationProperties(prefix = "my.book")
public class BookInfo {

    bname;

    bcost;

    List<String> authors;

    Set<String> authors;

    authors;

```

private Integer

private String

private boolean

private String

private int

//private

//private

private String[]

```
Map<String,String> versions;
```

private

```
Publisher pob; // HAS-A Variable
```

private

```
}
```

2. application.yml

```
my:
```

```
  book:
```

```
    bname: science
```

```
    authors:
```

```
      - HC VERMA
```

```
      - XYZ
```

```
      - PQR
```

```
      - HC VERMA
```

```
  versions:
```

```
    v1: E1
```

```
    v2: E2
```

```
    v3: E3
```

```
  pob:
```

```
    pid: 90012
```

```
    code: PC12345
```

```
    active: true
```

3. Runner class

```
BookRunner.java
```

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import com.app.bean.BookInfo;
```

```
@Component
```

```
public class BookRunner implements CommandLineRunner{
```

```
BookInfo bookInfo;
```

```
run(String... args) throws Exception {
```

```
    tln(bookInfo);
```

```
    intln(bookInfo.getAuthors().getClass().getName());
```

```
}
```

```
=====
```

```
output
```

```
BookInfo(bname=science, bcost=400, authors=[HC VERMA, XYZ, PQR, HC VERMA], versions={v1=E1, v2=E2, v3=E3}, pob=Publisher(pid=90012, code=PC12345, active=true))
```

```
-----
```

```
class Author{
```

```
}
```

```
class Book{
```

```
@Autowired
```

```
private
```

```
@Override
```

```
public void
```

```
System.out.prin
```

```
//System.out.pr
```

```
}
```

```
String name;
```

```
String addr;
```

authors; // multiple authors objects

```
Map<String,Authro> authors: /multiple author objects
}
```

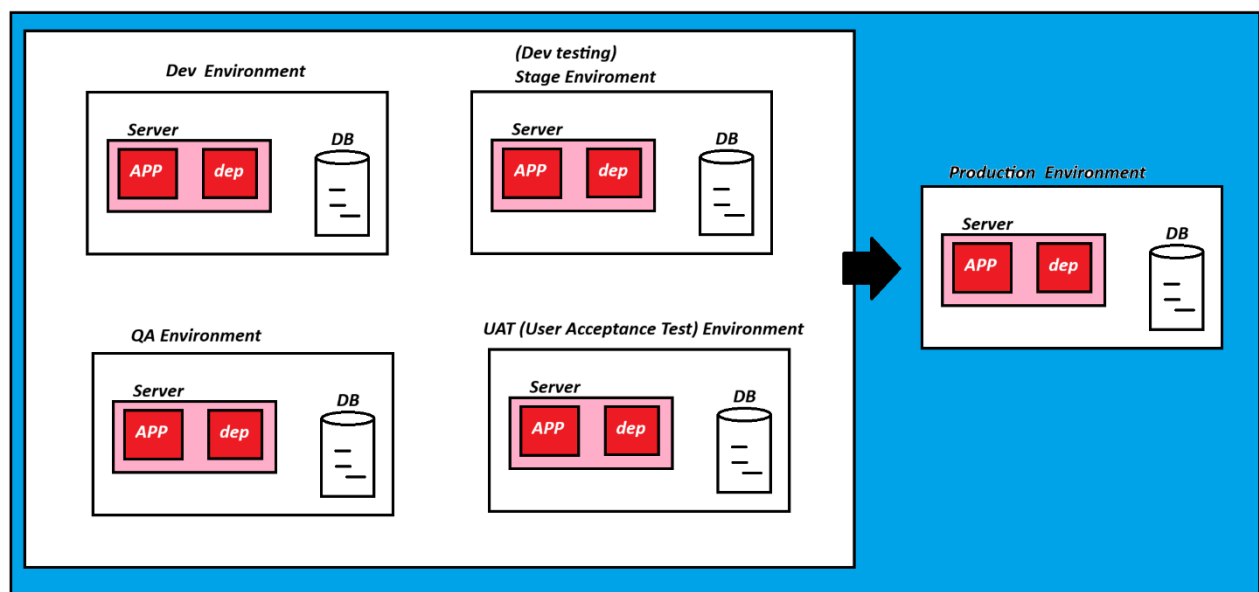
YAML File input must be given.

Environments:-

An environment is a separate setup or configuration where an application is developed, tested, or deployed for a specific purpose.

1. Dev Environment: Developer machine
2. QA Team : Testing Team
3. State Env : Developer Test
4. UAT Env : User Acceptance Test (Client Testing)
5. Production Env : Production Env (Actual server given to EndCustomer)

Diagram



profiles

Maintain multiple Properties/YAML files for multiple environments.

Based on Environment selected/running, matching properties/yaml file will be loaded

Rule Name : application-{profilename}.properties

Activate : (CMD) `--spring.profiles.active={profilename}`

examples:

Dev machine (default profile) `==> application.properties` or `application.yml`

UAT machine `==> application-uat.properties` or `application-uat.yml`

Production machine `==> application-prod.properties` or `application-prod.yml`

our application code remains same in almost all environments.

But properties may get varied. Ex: username and password or database.

=====

session 25

=====

Spring Boot Profiles

Maintain multiple Properties/YAML files for multiple environments.

based on Environment selected/running, matching properties/yaml file will be loaded.

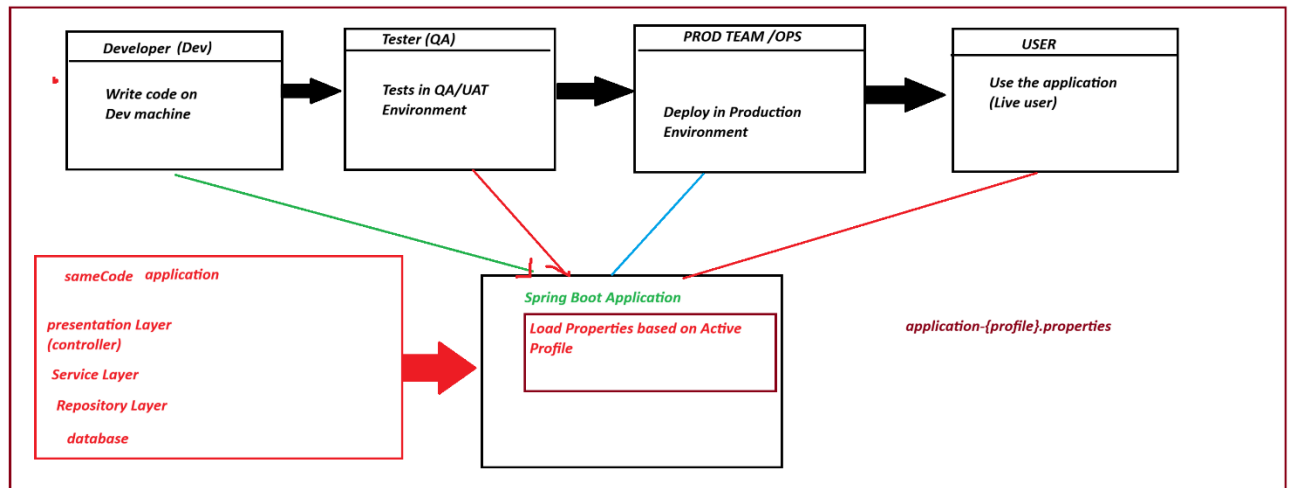
Name Rule : `application-{profileName}.properties`

Activate : (CMD) `--spring.profiles.active={profileName}`

How It works

(Steps)

1. Developer writes code (same code for all env)
2. Create multiple property files for different environments
3. Run application with Active profile using command line.
4. spring boot loads default file + matching profile file.
5. If any property missing in active profile -> take from default file.
6. Application connects to respective database.



example program

BootProfileEx-s25

Dependency Lombok

1. Spring bean

package com.app.bean;

import org.springframework.boot.context.properties.ConfigurationProperties;

import org.springframework.stereotype.Component;

import lombok.Data;

@Data

@Component

@ConfigurationProperties(prefix = "my.app.db")

public class MyDbCon {

private String driver;

private String url;

private String username;

private String password;

private String database;

}

2. Input files

application.properties

spring.application.name=BootProfileEx-s25

my.app.db.driver=Oracle

my.app.db.url=JDBC-ORCL

my.app.db.username=system

my.app.db.password=root

my.app.db.database=testdb

application-qa.properties

my.app.db.driver=MySQL

my.app.db.url=JDBC-MySQL

my.app.db.username=root

my.app.db.password=root

application-prod.properties

my.app.db.driver=Oracle

my.app.db.url=JDBC-ORCL-PROD

my.app.db.username=TEST

my.app.db.password=sample

my.app.db.database=testdb

3. Runner Class

package com.app.runner;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype.Component;

import com.app.bean.MyDbCon;

@Component

```
public class TestRunner implements CommandLineRunner {
```

@Autowired

```
private MyDbCon myDbCon;
```

@Override

```
public void run(String... args) throws Exception {
```

```
System.out.println(myDbCon);
```

```
}
```

```
}
```

output

The following 1 profile is active: "prod"

MyDbCon(driver=Oracle, url=JDBC-ORCL-PROD, username=TEST, password=sample, database=testdb)

=====

Right click main class code >Run As > Run Configuration.... > click Argument Tab> go to > program Arguments

Enter data like

--spring.profiles.active=qa

apply and Run

=====

Archive : Grouping all files as a single unit.

JAR (Java Archive)=> use for Standalone application

file1.java

file2.java

file3.java

file4.java

Archive

=====

WAR=> Web Archive

.class+html+css+javaScript+Images==> .war (web application)

=====

developer>.java>Compile>.class>.jar(build)

=====

Session 26

=====

Profiles using YAML Files

=> For properties file, we need define multiple files in case of Profiles concept. n-profiles=n-properties files;

YAML files too n-profile=n-YAML file

=> YAML even support single file for Multiple Profiles concept, using 3 dash symobls(---).

=> For every area we need to specify profile name (or) on which profile it should be loaded, using key.

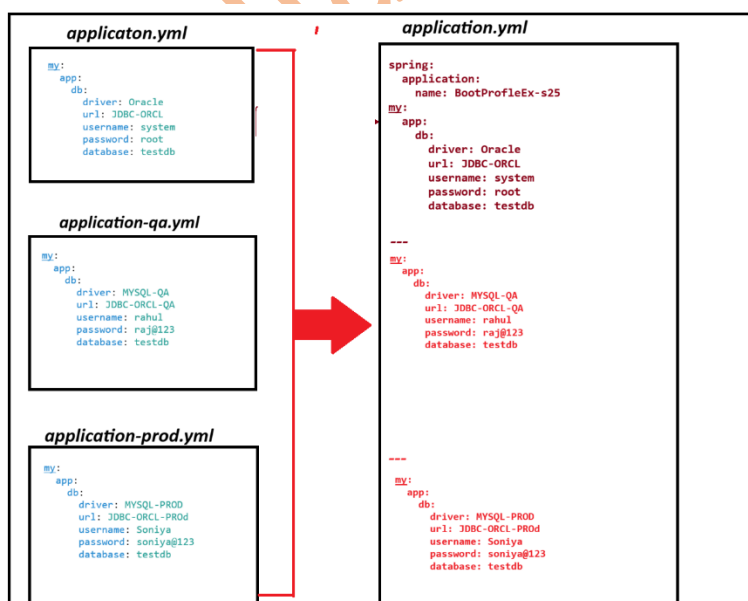
spring:

config:

active:

on-profiles:

- profile1
- profile2
- profile3



=====

Example project : BootProfileYAMLEx-s26

Dependency Lombok

1. Spring bean

package com.app.bean;

import org.springframework.boot.context.properties.ConfigurationProperties;

import org.springframework.stereotype.Component;

import lombok.Data;

@Data

@Component

@ConfigurationProperties(prefix = "my.app.db")

public class MyDbCon {

private String driver;

private String url;

private String username;

private String password;

private String database;

}

2. Input files

application.yml

spring:

application:

name: BootProfileEx-s25

my:

app:

db:

driver: Oracle

url: JDBC-ORCL

username: system

password: root

database: testdb

my:

app:

db:

driver: MYSQL-QA

url: JDBC-ORCL-QA

username: rahul

password: raj@123

database: testdb

spring:

config:

active:

on-profile: qa

my:

app:

db:

driver: MYSQL-Prod

url: JDBC-ORCL-prod

username: soniya

password: soniya@9999

database: testDb

spring:

config:

active:

on-profile: prod

=====

3. Runner Class

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import com.app.bean.MyDbCon;
```

```
@Component
```

```
public class TestRunner implements CommandLineRunner {
```

```
@Autowired
```

```
private MyDbCon myDbCon;
```

```
@Override
```

```
public void run(String... args) throws Exception {
```

```
System.out.println(myDbCon);
```

```
}
```

```
}
```

```
=====
```

output

```
MyDbCon(driver=MYSQL-Prod, url=JDBC-ORCL-prod, username=soniya, password=soniya@9999,
database=testDb)
```

```
=====
```

session 27

```
=====
```

@Autowired usecases

=> If Two Spring Beans are connected using Reference Type, (Two classes are connected using HAS-A relation)

then @Autowired annotation create link between them.

=> If spring container found one child object, then for @Autowired two object are linked.

=> If Spring container found zero child objects to link for Autowired then container throws exception:

UnsatisfiedDependencyException:

org.springframework.beans.factory.UnsatisfiedDependencyException:

Error creating
bean with name 'product': Unsatisfied dependency expressed through field 'vod': No qualifying bean of
type 'com.app.bean.Vendor' available: expected at least 1 bean which qualifies as autowire candidate.
Dependency annotations: {@org.springframework.beans.factory.annotation.Autowired(required=true)}

=> If we define @Autowired(required=false) then it will inform container if zero matches found. Please
keep it as null value.

=====

example project BootaAutowiredEx-s27

dependency Lombok

1. spring bean

package com.app.bean;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Component;

import lombok.ToString;

@Component

@ToString

public class Product {

@Autowired(required = false)

//@Autowired(required = true)

private Vendor vod; // HAS-A

}

2. spring bean

```
package com.app.bean;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.stereotype.Component;
```

```
import lombok.ToString;
```

```
@Component
```

```
@ToString
```

```
public class Vendor {
```

```
@Value("v1-xyz")
```

```
private String vname;
```

```
}
```

3. Runner class

```
package com.app.runner;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.stereotype.Component;
```

```
import com.app.bean.Product;
```

```
@Component
```

```
public class TestRunner implements CommandLineRunner {
```

```
@Autowired
```

```
private Product pob;
```


@Override

```
public void run(String... args) throws Exception {  
    System.out.println(pob);  
}
```

```
}
```

java Config : Two define multiple objects create obj for a pre-defined class

1. write one public class
2. Apply @Configuration (Input to Container)
3. Define one method for one object

```
public <ClassName> <objName>(){  
}
```

=====code=====

1. spring bean

```
package com.app.bean;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.beans.factory.annotation.Qualifier;
```

```
import org.springframework.stereotype.Component;
```

```
import lombok.ToString;
```

@Component

@ToString

```
public class Product {
```

```
//@Autowired(required = false)
```

```
//@Autowired(required = true)
```

```
@Autowired
//@Qualifier("vod2")
private Vendor vod; // HAS-A
```

```
}
```

2.config

```
package com.app.config;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.context.annotation.Primary;
```

```
import com.app.bean.Vendor;
```

```
@Configuration
```

```
public class MyAppConfig {
```

```
@Bean
```

```
@Primary
```

```
public Vendor vod1() {
```

```
Vendor v1=new Vendor();
```

```
v1.setVname("v1-abc");
```

```
return v1;
```

```
}
```

```
@Bean
```

```
public Vendor vod2() {
```

```
Vendor v1=new Vendor();
```

```
v1.setVname("v1-pqr");  
  
return v1;  
  
}  
  
}
```

spring bean

package com.app.bean;

```
import org.springframework.beans.factory.annotation.Value;
```

```
import lombok.Data;
```

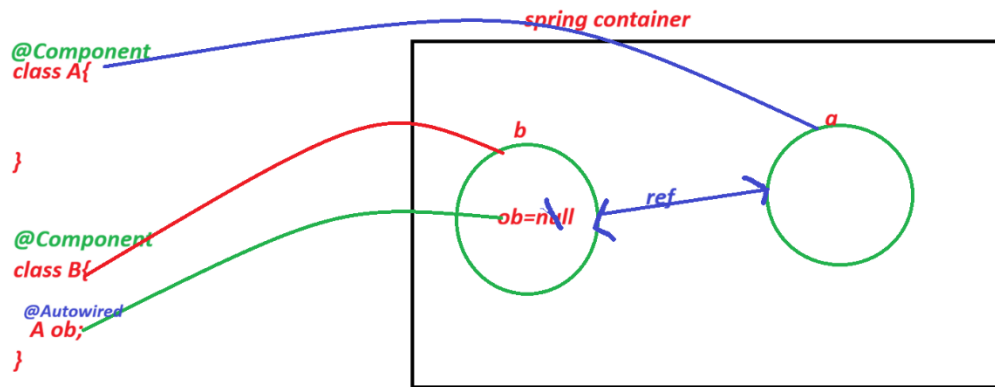
```
@Data
```

```
public class Vendor {
```

```
private String vname;
```

```
}
```

Diagram



✓ 0 objects found	@Autowired -> @Autowired(required=true) NoSuchBeanDefinitionException ✓ @Autowired(required=false)--> keep it as null ✓
1 obj found	If one child object is found, container will link them ✓ <u>(No Error, No Exception, we get output)</u>
1 obj found	If two or more child objects found then NoUniqueBeanDefinitionException solution: 1. byName: we can modify one child bean name as Ref variable name 2. manual Select **** use @Qualifier("objname") will search and link that child obj 3. Default obj Recommended obj @Primary says if multiple child beans found then select this by default

session 28

=====

Download and Install MySQL download

Link <https://dev.mysql.com/downloads/installer/>

=====

ORM (Object Relation Mapping)

=====

JDBC :

JDBC Stands for Java Database Connectivity.

=> JDBC is an API given by java.

=> used to perform database Operations.

=> JDBC works mainly on SQL Concept. SQL contains all primitive data.

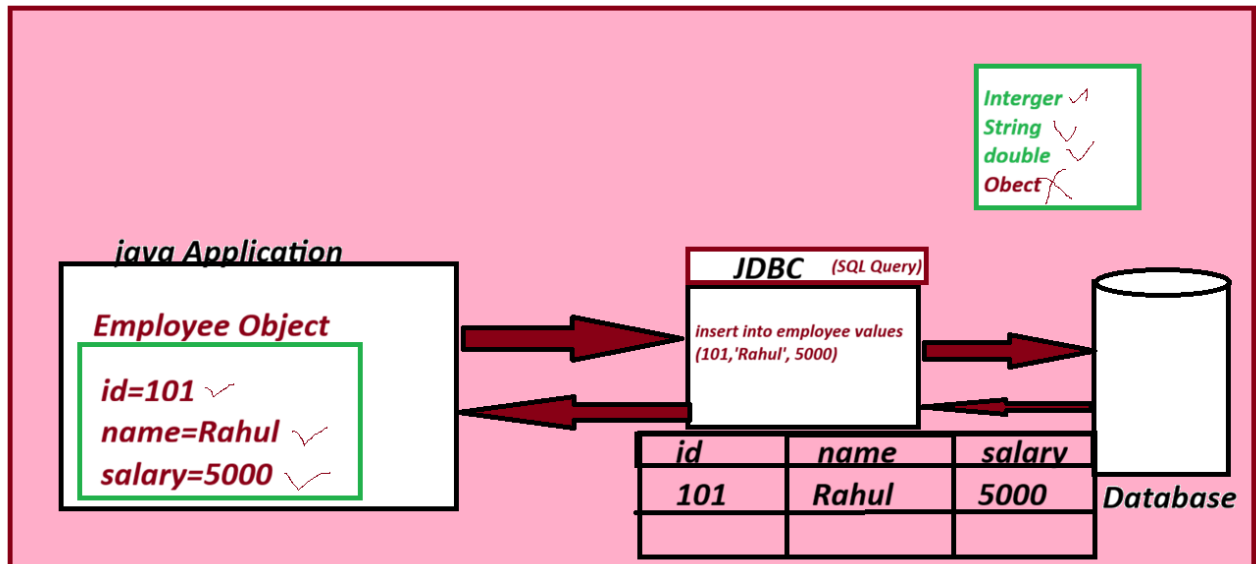
=> Even we have data in Object, should be converted into primitives then set to SQL.

employee object-> eob=> (eid=10,ename="A",esal=300.0);

problem:

we need to write SQL for every operation.

More code, more time, more errors.



ORM (Object Relation Mapping):

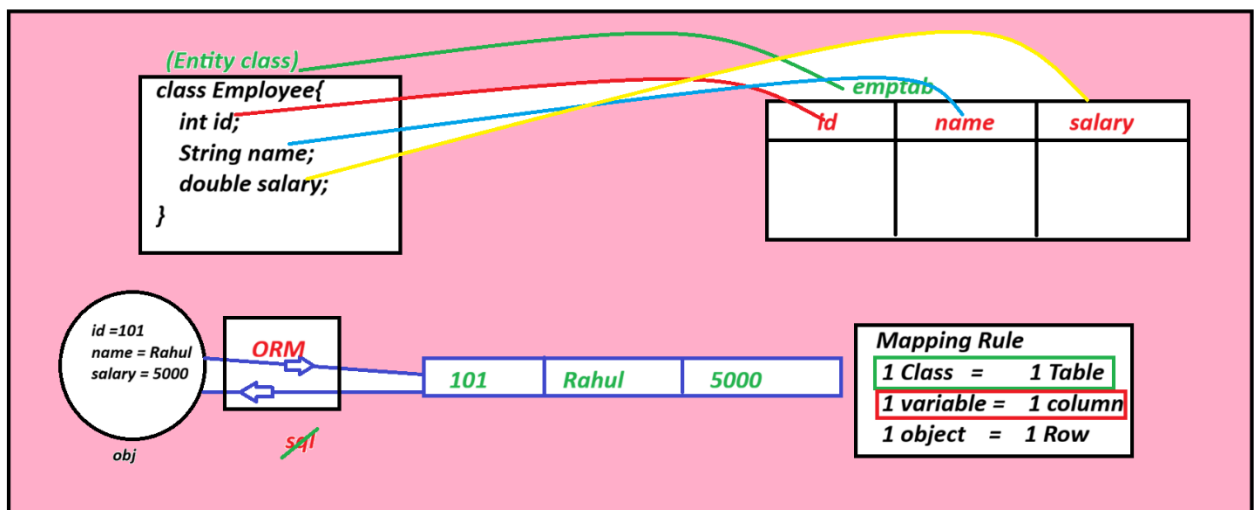
=> ORM stands for Object Relational Mapping.

=> It allows database operations using java Objects instead of writing SQL manually.

=> Writing SQL for every operation is repetitive and time-consume. ORM reduces code and improves maintainability.

=> we create an object and call methods like save(), update(), delete(), or get().

=> ORM automatically converts the object into SQL.



=====

JPA(Java Persistence API)

=====

=> JPA is a specification provided by Sun/Oracle to perform database operation using ORM Concept.

=> JPA defines standard rules so that application are independent of a specific ORM vendor.

=> we use JPA Annotation in our entity classes

@Entity

@Table

@Id

@Column

package javax.persistence

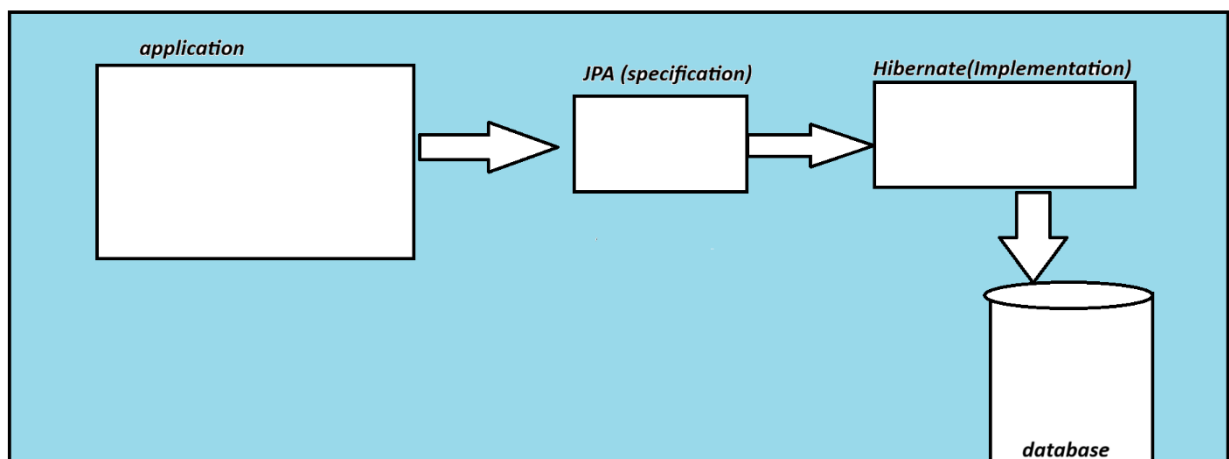
new version jakarta.persistence

Hibernate :

=> Hiberante is a JPA vendor that implements the JPA specification.

=> JPA is only a specification. It cannot execute database operations by itself. A vendor implmentation like Hibernate , EclipseLink, OpenJPA, IBatis etc..

=> Hibernate reads JPA annotations, generates SQL queires automatically, and communicates with database.



@Entity

class Employee{

@Id

```
Integer id;  
String name;  
double salary;  
}
```

=====

(Q) What is the diff b/w ORM , JPA and Hibernate?

=> ORM ==> Theory / Concept

=> JPA ==> API (set of classess, interfaces, Enum, Annotation..) etc.

=> Hibernate ==> Implements ORM/JPA. It is JPA vendor.

(Q) can we write ORM apps only using JPA without any vendor API?

=> No, we have to used any one vendor api.

(Q) can we write ORM apps only using Hibernate without JPA?

=> Yes.

=====

session 30

=====